



For Instructor Use Only.
This document should not be distributed.

Oracle AI Vector Search Fundamentals

Student Guide
D1109439GC10

Copyright © 2025, Oracle and/or its affiliates.

Disclaimer

This document contains proprietary information and is protected by copyright and other intellectual property laws. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

Restricted Rights Notice

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

Trademark Notice

Oracle®, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

Third-Party Content, Products, and Services Disclaimer

This documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

1001242025

For Instructor Use Only.
This document should not be distributed.

Contents

I Course Overview

Course Outline I-2

Target Audience I-6

Learning Outcomes I-7

What's next? I-9

1 Overview of Oracle AI Vector Search

Objectives 1-2

VECTOR Data Type 1-3

Vector Embeddings 1-5

Similarity Search 1-7

Vector Embedding Models 1-9

Import Embedding Models 1-11

Summary 1-13

2 Why use Oracle AI Vector Search?

Objectives 2-2

Benefits 2-3

Benefits of Oracle AI Vector Search 2-4

Examples 2-5

Summary 2-6

3 Oracle AI Vector Search Workflow

Objectives 3-2

Generate Vector Embeddings 3-3

Generate Embeddings Examples 3-5

Example: Load ONNX Model into the DB 3-6

Example: Generate the Embedding 3-7

Store Vector Embeddings 3-8

Vector Indexes 3-10

Query Data with Similarity Searches 3-12

The Complete Workflow 3-14

Summary 3-15

For Instructor Use Only.
This document should not be distributed.

4 Running Basic Queries on Vectors

- Objectives 4-2
- Basic Queries 4-3
- Basic Queries: Comparison Operations 4-7
- Summary 4-9

5 Vector Indexes and Memory

- Objectives 5-2
- Vector Indexes 5-3
- Vector Index Categories 5-5
- Vector Pool in the SGA 5-7
- Vector Pool 5-8
- Vector Pool: Other Considerations 5-9
- Memory Considerations 5-10
- Vector Storage 5-12
- Memory Considerations: In-Memory Neighbor Graph Indexes 5-13
- Memory Considerations: Sample Calculation 5-14
- Summary 5-15

6 DML Operations on Vectors

- Objectives 6-2
- Create a Table with a Vector Column 6-3
- VECTOR Data Type 6-4
- Declaration Formats and Explanation 6-6
- Vector DML 6-7
- Vector DML Using SQL*Loader 6-10
- Summary 6-14

7 Vector DDL

- Objectives 7-2
- Tables with Different Vector Formats 7-3
- Tables with Different Vector Formats: Example 7-5
- Tables with Different Vector Formats: Insert Example 7-6
- DDL Operations on Vectors 7-7
- Prohibited Operations 7-10
- VECTOR Data Type Restrictions 7-11
- Summary 7-14

8 Creating and Finding the Nearest Vectors

- Objectives 8-2
- Vector Constructor 8-3

- Vector Constructor Examples 8-5
- Vector Distance 8-6
- Vector Distance Operand 8-7
- Vector Distance Metrics 8-8
- Euclidean and Euclidean Squared Distances 8-9
- Cosine Similarity 8-10
- Dot Product Similarity 8-11
- Manhattan Distance 8-12
- Hamming Similarity 8-13
- Vector Distance Examples 8-14
- Shorthand Operators For Distances 8-15
- Summary 8-18

9 Finding the Closest Vectors

- Objectives 9-2
- Exact Similarity Search 9-3
- Euclidean 9-6
- Euclidean Squared Distance 9-7
- Approximate Similarity Search 9-8
- Approximate Similarity Search or Exact Similarity Search? 9-10
- Approximate Similarity Search 9-11
- Approximate Similarity Search: HNSW 9-12
- Approximate Similarity Search: IVF 9-13
- Multi-Vector Similarity Search 9-14
- Summary 9-19

10 Narrowing Search Results

- Objectives 10-2
- Attribute Filtering 10-3
- Summary 10-13

11 Testing Other Distance Functions

- Objectives 11-2
- Other Distance Functions 11-3
- Other Vector Distance Functions 11-4
- L1_DISTANCE 11-5
- L2_DISTANCE 11-6
- COSINE_DISTANCE 11-7
- INNER_PRODUCT 11-8
- Summary 11-10

12 Testing Other Vector Functions

Objectives 12-2

Other Vector Functions 12-3

Vector Constructors 12-5

Vector Serializers 12-7

Vector Norm 12-9

Vector Dimension Count 12-10

Vector Dimension Format 12-11

Summary 12-13

13 Course Conclusion

Learning Summary 13-2

Thank You! 13-4

For Instructor Use Only.
This document should not be distributed.

Course Overview

Oracle AI Vector Search Fundamentals



For Instructor Use Only.
This document should not be distributed.

Course Outline

Overview of Oracle AI Vector Search

- > VECTOR Data Type
- > Vector Embeddings
- > Similarity Search
- > Vector Embedding Models
- > Import Embedding Models

Why Use Oracle AI Vector Search

- > Benefits

Oracle AI Vector Search Workflow

- > Generate Vector Embeddings
- > Store Vector Embeddings
- > Vector Indexes
- > Query Data with Similarity Searches



For Instructor Use Only.
This document should not be distributed.

Course Outline

Running Basic Queries on Vectors

- > Exact Similarity Search
- > Approximate Similarity Search
- > Multi-Vector Similarity Search

Vector Indexes and Memory

- > Vector Indexes
- > Memory Considerations

DML Operations on Vectors

- > Create a Table with a Vector Column
- > Vector Inserts, Updates, and Deletes



For Instructor Use Only.

This document should not be distributed.

Course Outline

Vector DDL

- > Tables with Different Vector Formats
- > DDL Operations on Vectors
- > Prohibited Operations

Create and Find Nearest Vectors

- > Vector Distance

Find Closest Vectors

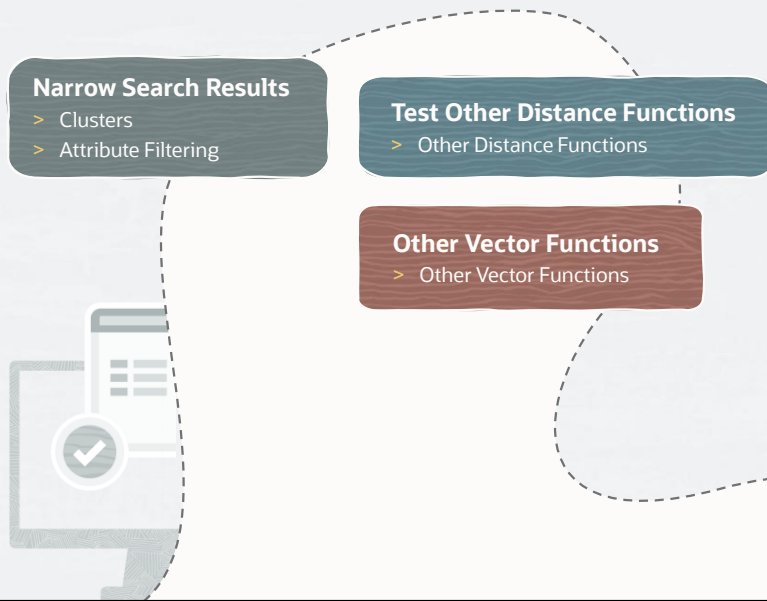
- > Similarity Search



For Instructor Use Only.

This document should not be distributed.

Course Outline



For Instructor Use Only.
This document should not be distributed.

Target Audience



Beginners



Developers

The target audience of this course includes:

- Beginners that are just starting out with databases
- Developers that are required to maintain their development databases

Learning Outcomes

Vector Search
Overview



Benefits



VECTOR Datatype



Vector Embedding
Models



Vector Workflow



For Instructor Use Only.
This document should not be distributed.

Learning Outcomes

Basic Queries
on Vectors



Vector Indexes



Vector Memory



Vector DML
and DDL



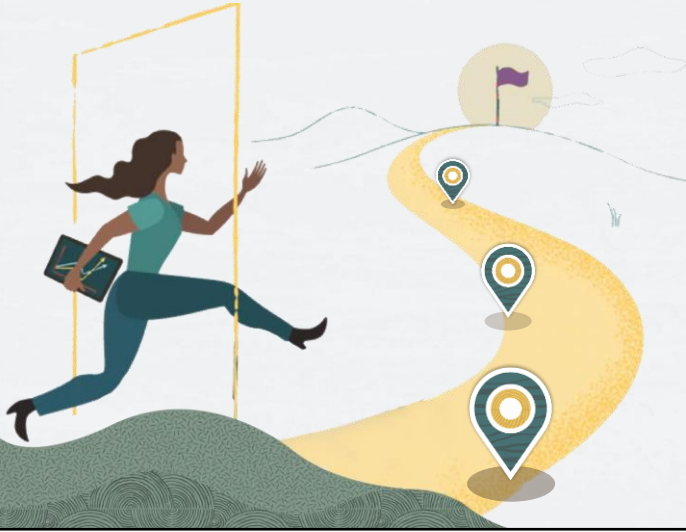
Vector Functions
and Filtering



For Instructor Use Only.

This document should not be distributed.

What's next?



**Oracle Database 23ai:
SQL Workshop**

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Overview of Oracle AI Vector Search



For Instructor Use Only.

This document should not be distributed.

Objectives



VECTOR Data Type

Vector Embeddings

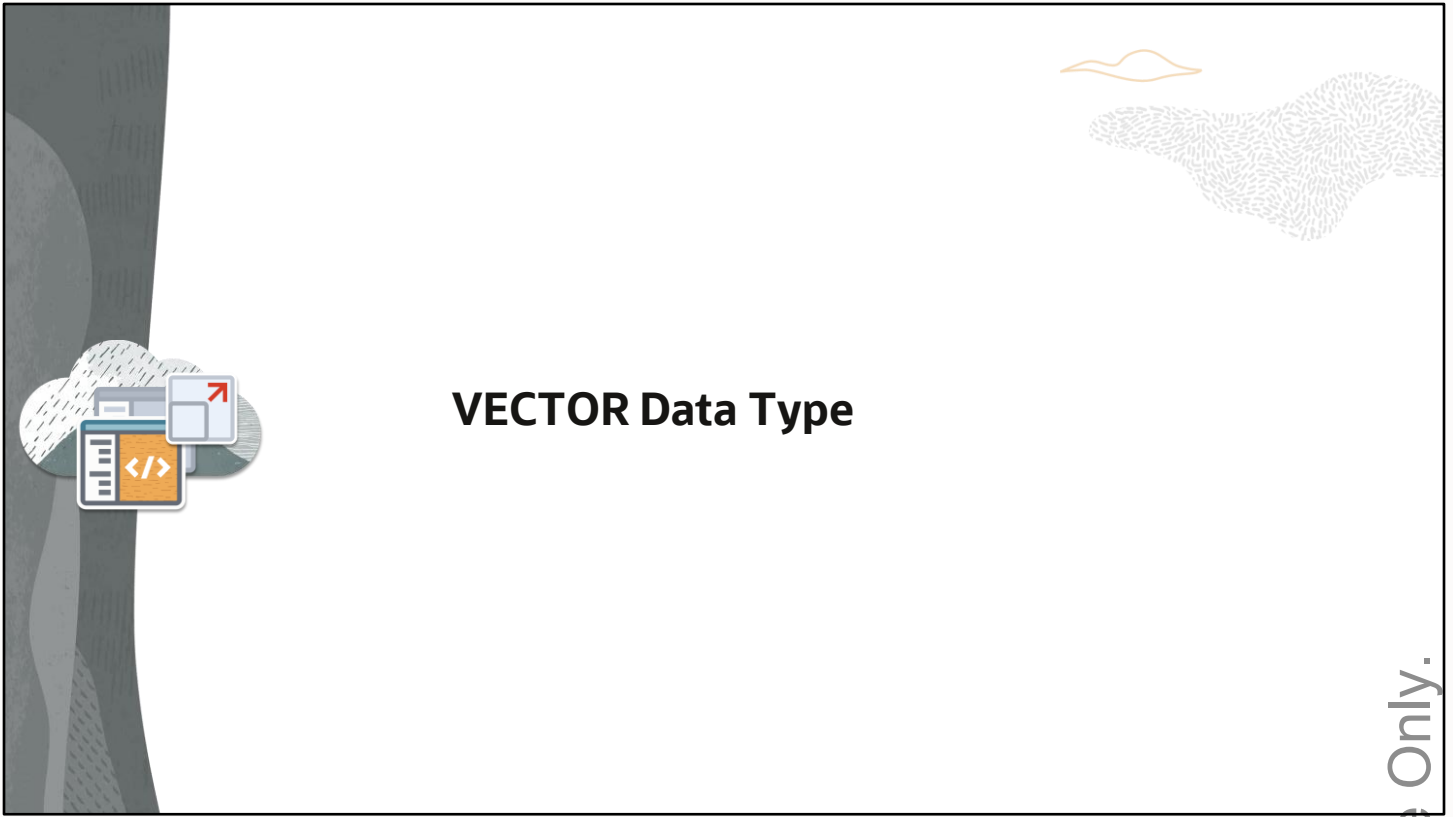
Similarity Search

Vector Embedding Models

Import Embedding Models

In this lesson, you gain an understanding of Oracle AI Vector Search including:

- Vector Data Type
- Vector Embeddings
- Similarity Search
- Vector Embedding Models
- Importing Embedding Models



VECTOR Data Type

For Instructor Use Only.
This document should not be distributed.

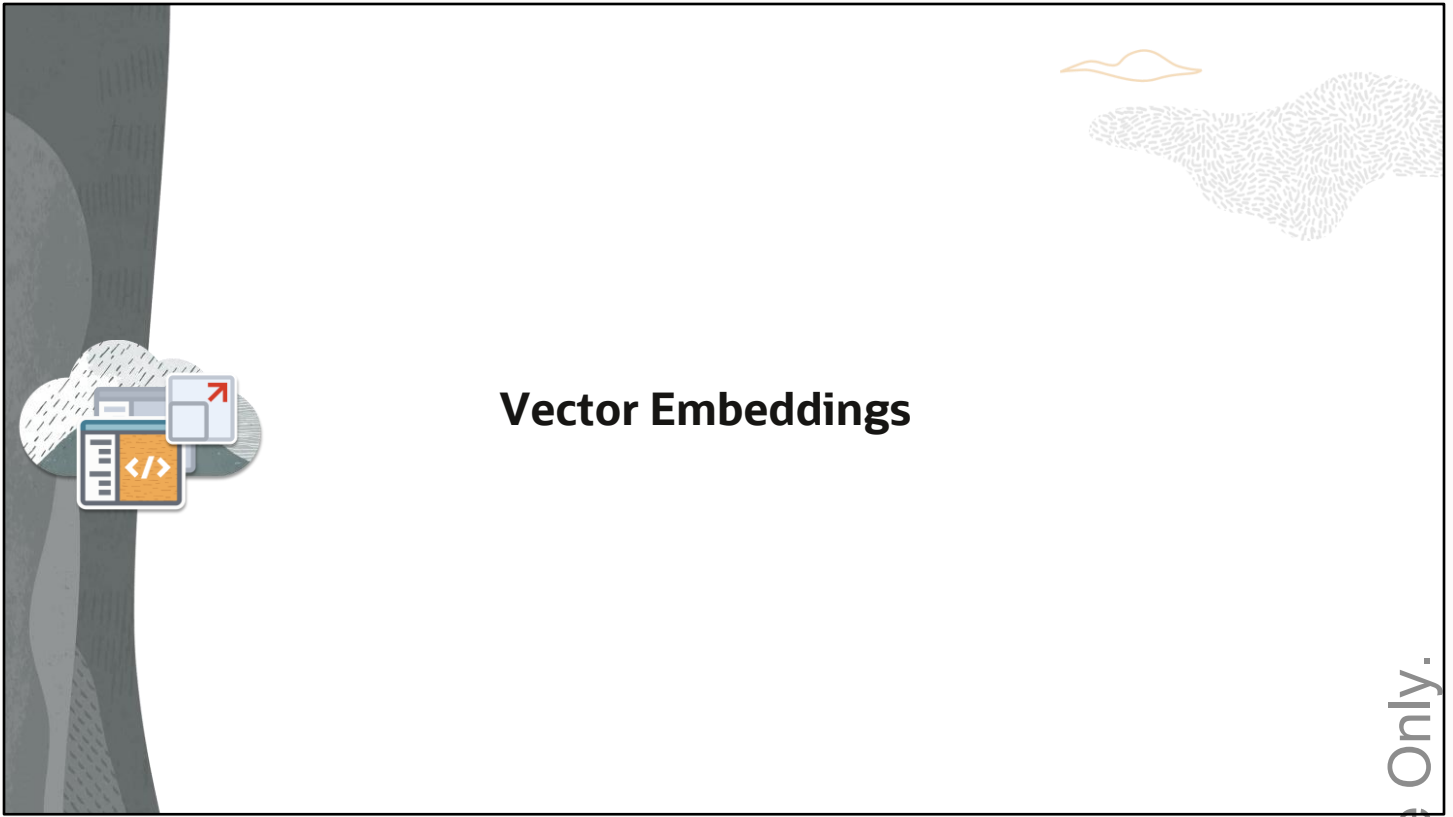
VECTOR Data Type

- > Introduced in Oracle Database 23ai
- > Stores vector embeddings alongside other business data



Oracle AI Vector Search is designed for Artificial Intelligence (AI) workloads and allows you to query data based on semantics, rather than keywords.

Oracle Database 23ai introduces the VECTOR data type, providing the foundation to store vector embeddings alongside business data in the database. Using embedding models, you can transform unstructured data into vector embeddings that can then be used for semantic queries on business data.



Vector Embeddings

For Instructor Use Only.
This document should not be distributed.

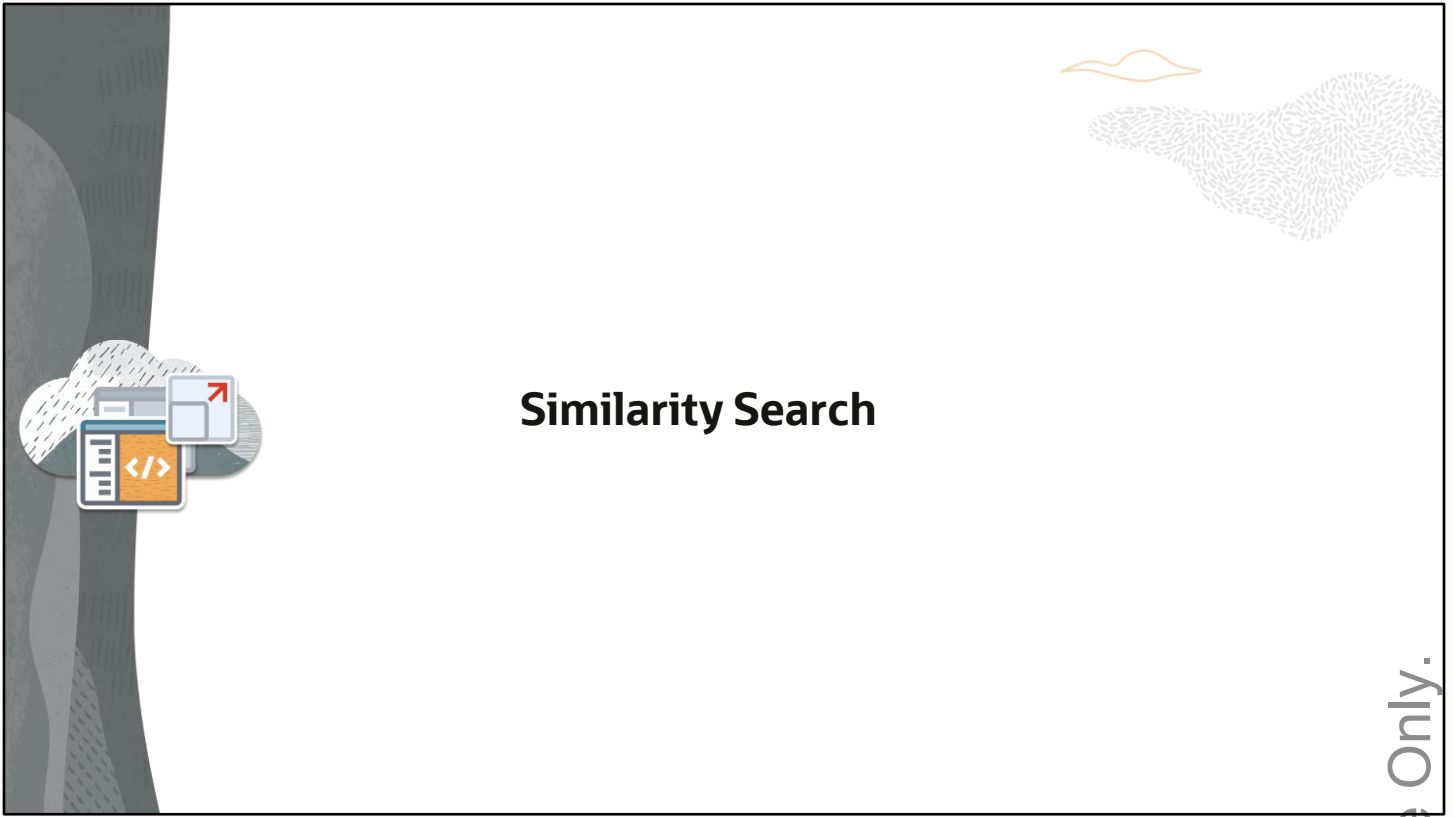
Vector Embeddings

- > Mathematical representations of data points
- > Based on meaning and context
- > Can be used to represent almost any type of data
- > Used in proximity searches



Oracle AI Vector Search stores vector embeddings, which are mathematical vector representations of data points. These vector embeddings describe the semantic meaning behind content such as words, documents, audio tracks, or images. As an example, while doing text-based searches, vector search is often considered better than keyword search as vector search is based on the meaning and context behind the words and not the actual words themselves. This vector representation translates the semantic similarity of objects, as perceived by humans, into proximity in a mathematical vector space. This vector space usually has multihundreds, if not thousands, of dimensions. Put differently, vector embeddings are a way of representing almost any kind of data, such as text, images, videos, users, or music as points in a multidimensional space where the locations of those points in space, and proximity to others, are semantically meaningful.

If you've ever used applications such as voice assistants, chatbots, language translators, recommendation systems, anomaly detection, or video search and recognition, you've implicitly used vector embedding features.

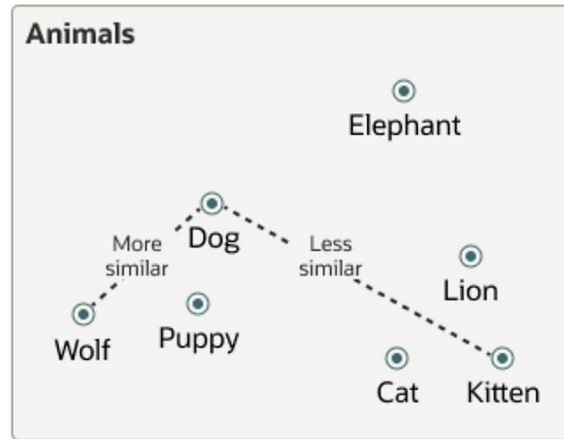


Similarity Search

For Instructor Use Only.
This document should not be distributed.

Similarity Search

- > Vector data is usually:
 - Unevenly distributed
 - Clustered
- > Search for nearest neighbors:
 - Ordered list by rank
- > Searches can be:
 - Approximate
 - Specific

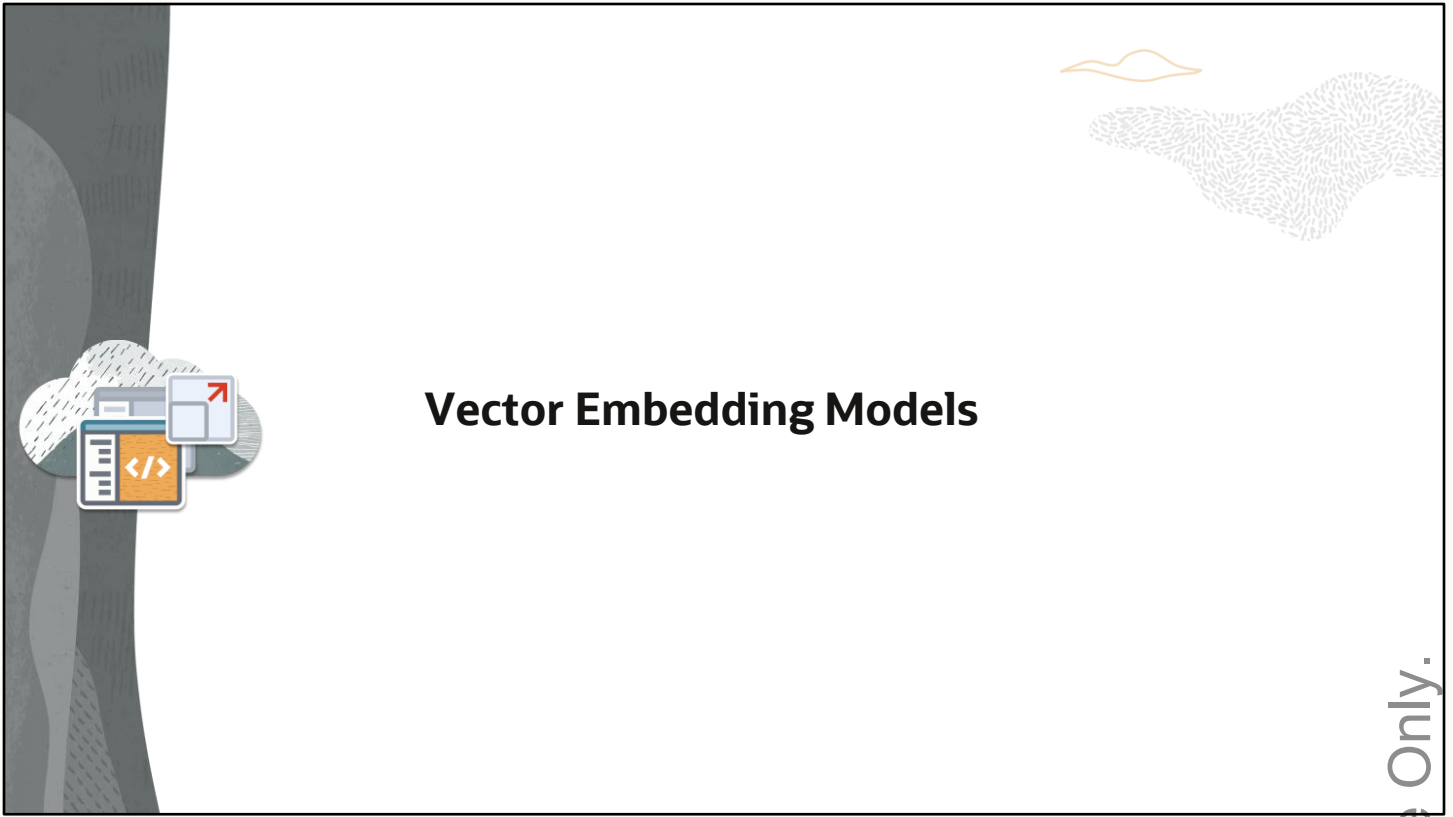


Searching for semantic similarity in a data set is now equivalent to searching for nearest neighbors in a vector space instead of using traditional keyword searches using query predicates. As an example in the above diagram, the distance between dog and wolf in this vector space is shorter than the distance between dog and kitten. In this space, a dog is more similar to a wolf than it is to a kitten.

Vector data tends to be unevenly distributed and clustered into groups that are semantically related. Doing a similarity search based on a given query vector is equivalent to retrieving the K-nearest vectors to your query vector in your vector space. Basically, you need to find an ordered list of vectors by ranking them, where the first row in the list is the closest or most similar vector to the query vector, the second row in the list is the second closest vector to the query vector, and so on. When doing a similarity search, the relative order of distances is what really matters rather than the actual distance.

Similarity searches tend to get data from one or more clusters depending on the value of the query vector and the fetch size.

Approximate searches using vector indexes can limit the searches to specific clusters, whereas exact searches visit vectors across all clusters.

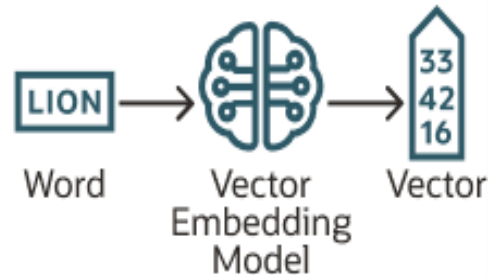


Vector Embedding Models

For Instructor Use Only.
This document should not be distributed.

Vector Embedding Models

- > Quantify features or dimensions
- > Models can be:
 - Pretrained open source
 - Based on your own data set



One way of creating vector embeddings could be to use someone's domain expertise to quantify a predefined set of features or dimensions such as shape, texture, color, sentiment, and many others, depending on the object type with which you're dealing. However, the efficiency of this method depends on the use case and is not always cost effective.

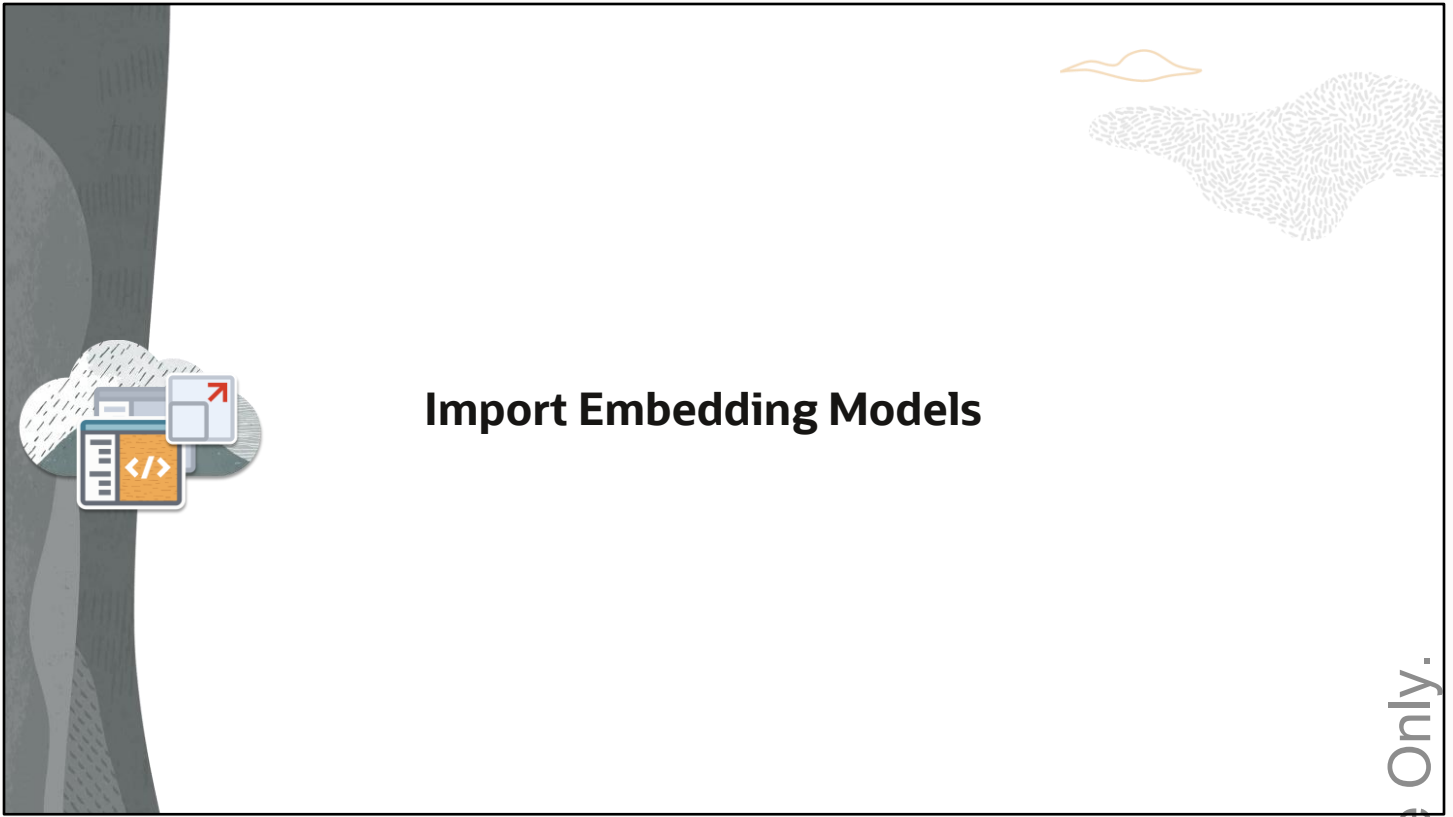
Instead, vector embeddings are created via neural networks. Most modern vector embeddings use a transformer model, as illustrated in the diagram above, but convolutional neural networks can also be used.

Depending on the type of your data, you can use different pretrained, open-source models to create vector embeddings. For example:

- For textual data, sentence transformers transform words, sentences, or paragraphs into vector embeddings.
- For visual data, you can use Residual Network (ResNet) to generate vector embeddings.
- For audio data, you can use the visual spectrogram representation of the audio data to fall back into the visual data case.

Each model also determines the number of dimensions for your vectors. For example:

- Cohere's embedding model embed-english-v3.0 has 1024 dimensions.
- OpenAI's embedding model text-embedding-3-large has 3072 dimensions.
- Hugging Face's embedding model all-MiniLM-L6-v2 has 384 dimensions
- Of course, you can always create your own model that is trained with your own data set.



Import Embedding Models

For Instructor Use Only.
This document should not be distributed.

Import Embedding Models

- > Generate vector embeddings:
 - Outside the Oracle DB
 - Within the Oracle DB (ONNX)



Although you can generate vector embeddings outside the Oracle Database using pretrained open-source embeddings models or your own embeddings models, you also have the option to import those models directly into the Oracle Database if they are compatible with the Open Neural Network Exchange (ONNX) standard. Oracle Database implements an ONNX runtime directly within the database. This allows you to generate vector embeddings directly within the Oracle Database using SQL.

For Instructor Use Only.

This document should not be distributed.

Summary



VECTOR Data Type

Vector Embeddings

Similarity Search

Vector Embedding Models

Import Embedding Models

In this lesson, you have learned about Oracle AI Vector Search including:

- Vector Data Type
- Vector Embeddings
- Similarity Search
- Vector Embedding Models
- Importing Embedding Models

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Why use Oracle AI Vector Search?



For Instructor Use Only.

This document should not be distributed.

Objectives



Benefits

In this lesson, you gain an understanding of the benefits of using Oracle AI Vector Search.



For Instructor Use Only.
This document should not be distributed.

Benefits of Oracle AI Vector Search

- > Single system for:
 - Semantic search on unstructured data
 - Relational search on business data
 - Eliminates data fragmentation
- > Also supports:
 - Retrieval Augmented Generation (RAG)



One of the biggest benefits of Oracle AI Vector Search is that semantic search on unstructured data can be combined with relational search on business data in one single system.

This is not only powerful but also significantly more effective because you don't need to add a specialized vector database, eliminating the pain of data fragmentation between multiple systems.

These new capabilities also support [Retrieval Augmented Generation \(RAG\)](https://www.oracle.com/artificial-intelligence/generative-ai/retrieval-augmented-generation-rag/) (<https://www.oracle.com/artificial-intelligence/generative-ai/retrieval-augmented-generation-rag/>), a breakthrough generative AI technique that combines large language models (LLMs) and private business data to deliver responses to natural language questions. RAG provides higher accuracy and avoids having to expose private data by including it in the LLM training data.

Note: RAG is not covered in this course.

Examples

- > Create a table:

```
CREATE TABLE house_for_sale (house_id NUMBER, price NUMBER, city  
VARCHAR2(400), house_photo BLOB, house_vector VECTOR);
```

- > Query the table:

```
SELECT house_photo, city, price FROM house_for_sale  
WHERE price <= :input_price AND city = :input_city  
ORDER BY VECTOR_DISTANCE(house_vector, :input_vector);
```

As an example, suppose you use an application that allows you to find a house that is similar to a picture you took of one you like that is located in your preferred area for a certain budget. Finding a good match in this case requires combining a semantic picture search with searches on business data.

In conjunction with Oracle Database 23ai, Oracle Exadata System Software release 24.1.0 introduces AI Smart Scan, a collection of Exadata-specific optimizations capable of improving the performance of various AI vector query operations by orders of magnitude.

AI Smart Scan automatically accelerates Oracle Database 23ai AI Vector Search with optimizations that deliver low-latency parallelized scans across massive volumes of vector data. AI Smart Scan processes vector data at memory speed, leveraging ultra-fast Exadata RDMA Memory (XRMEM) and Exadata Smart Flash Cache in the Exadata storage servers, and performs vector distance computations and top-K filtering at the data source, avoiding unnecessary network data transfer and database server processing.

Summary



Benefits

In this lesson, you have gained an understanding of the benefits of using Oracle AI Vector Search

Oracle AI Vector Search Workflow



For Instructor Use Only.

This document should not be distributed.

Objectives



Generate Vector Embeddings

Store Vector Embeddings

Vector Indexes

Query Data with Similarity Searches

In this lesson, you gain an understanding of the Oracle AI Vector Search workflow.

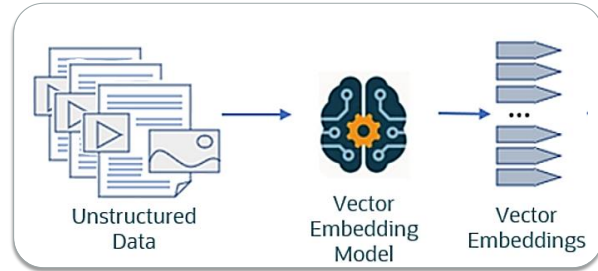


Generate Vector Embeddings

For Instructor Use Only.
This document should not be distributed.

Generate Vector Embeddings

- > Generate vector embeddings from your data:
 - Outside the DB
 - Within the DB



Primary workflow steps:

1. **Generate Vector Embeddings from Your Unstructured Data.** You can perform this step either outside or within Oracle Database.

Generate Embeddings Examples

- > Convert Text String to Embedding
- > Convert File to Text to Chunks to Embeddings
- > Convert File to Embeddings
- > Generate and Use Embeddings for End-to-End Search

You must generate vector embeddings from your unstructured data either outside or within Oracle Database.

To get vector embeddings, you can either use ONNX embedding machine learning models or access third-party REST APIs.

Import Pretrained Models in ONNX Format for Vector Generation Within the Database

You can download pretrained embedding machine learning models, convert them into ONNX format if they are not already in ONNX format, import the ONNX format models into Oracle Database, and generate vector embeddings from your data within the database.

Convert Pretrained Models to ONNX Format

OML4Py enables the use of text transformers from Hugging Face by converting them into ONNX format models. OML4Py also adds the necessary tokenization and post-processing. The resulting ONNX pipeline is then imported into the database and can be used to generate embeddings for AI Vector Search.

Generate Vector Embeddings Using Vector Utilities Leveraging Third-Party REST APIs

Oracle AI Vector Search offers vector utilities (SQL and PL/SQL tools) to automatically generate vector embeddings from your unstructured data.

Example: Load ONNX Model into the DB

```
EXECUTE DBMS_VECTOR.LOAD_ONNX_MODEL (  
    'DM_DUMP',  
    'my_embedding_model.onnx',  
    'doc_model');
```

You can use DBMS_DATA_MINING.import_onnx_model PL/SQL API to load ONNX models into the database.

Once you have the ONNX models loaded into the database, you can put them to use to convert data into vectors. You can leverage the vector_embedding SQL function to create vector embeddings for your data.

As an example, create a vector embedding of an overview paragraph (your data point) using the doc_model ONNX model loaded into the database. Then store this vector into a vector column in your database table.

Once you have the vectors stored in the database, you can now compare vectors, perform semantic search by calculating the vector distance between them using the vector_distance SQL function.

The example above uses DBMS_VECTOR to load the ONNX model into the Database.

Example: Generate the Embedding

```
SELECT TO_VECTOR(VECTOR_EMBEDDING(doc_model USING 'hello' as data)) AS embedding;
```

Use the ONNX model to generate the embedding.

For Instructor Use Only.
This document should not be distributed.

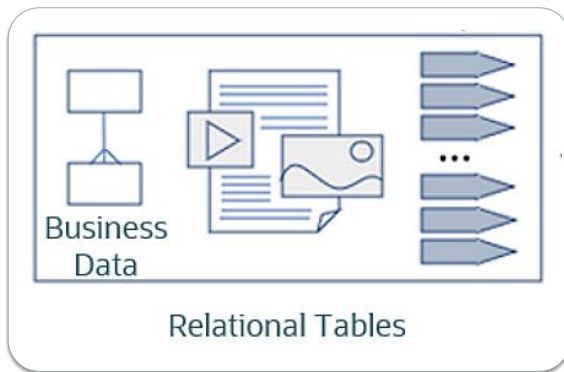


Store Vector Embeddings

For Instructor Use Only.
This document should not be distributed.

Store Vector Embeddings

- > Create column(s) of VECTOR data type.



Primary workflow steps:

1. **Generate Vector Embeddings from Your Unstructured Data.** You can perform this step either outside or within Oracle Database.
2. **Store Vector Embeddings, Unstructured Data, and Relational Business Data in Oracle Database.** You store the resulting vector embeddings and associated unstructured data with your relational business data in Oracle Database.

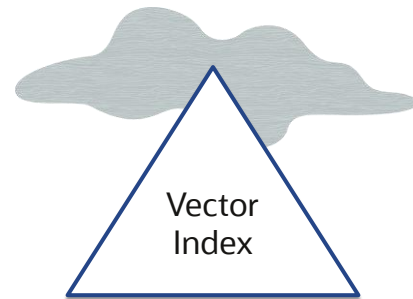


Vector Indexes

For Instructor Use Only.
This document should not be distributed.

Vector Indexes

- > Create vector indexes.
 - Used for huge vector spaces



Primary workflow steps:

1. **Generate Vector Embeddings from Your Unstructured Data.** You can perform this step either outside or within Oracle Database.
2. **Store Vector Embeddings, Unstructured Data, and Relational Business Data in Oracle Database.** You store the resulting vector embeddings and associated unstructured data with your relational business data in Oracle Database.
3. **Create Vector Indexes.** You may want to create vector indexes on your vector embeddings. This is beneficial for running similarity searches over huge vector spaces.

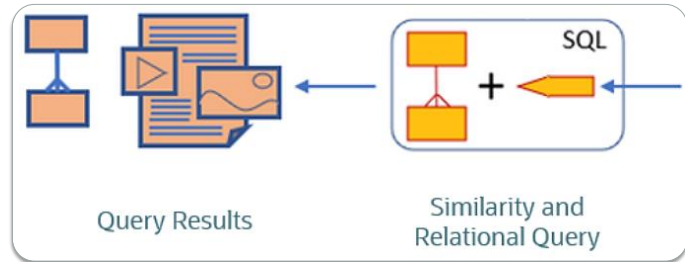


Query Data with Similarity Searches

For Instructor Use Only.
This document should not be distributed.

Query Data with Similarity Searches

- > Native SQL Operations
- > Combine:
 - Similarity searches
 - Relational searches

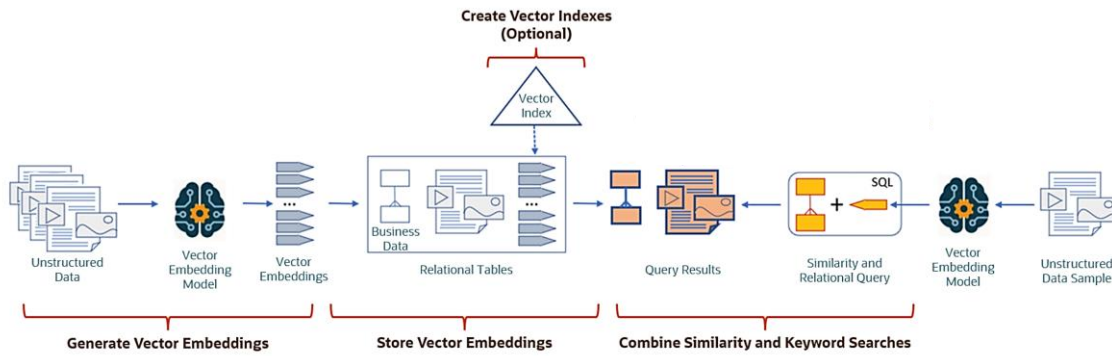


Primary workflow steps:

1. **Generate Vector Embeddings from Your Unstructured Data.** You can perform this step either outside or within Oracle Database.
2. **Store Vector Embeddings, Unstructured Data, and Relational Business Data in Oracle Database.** You store the resulting vector embeddings and associated unstructured data with your relational business data in Oracle Database.
3. **Create Vector Indexes.** You may want to create vector indexes on your vector embeddings. This is beneficial for running similarity searches over huge vector spaces.
4. **Query Data with Similarity Searches.** You can then use Oracle AI Vector Search native SQL operations to combine similarity with relational searches to retrieve relevant data.

The Complete Workflow

- > Optional:
 - Use with generative LLM for RAG interface.



Primary workflow steps:

1. **Generate Vector Embeddings from Your Unstructured Data.** You can perform this step either outside or within Oracle Database.
2. **Store Vector Embeddings, Unstructured Data, and Relational Business Data in Oracle Database.** You store the resulting vector embeddings and associated unstructured data with your relational business data in Oracle Database.
3. **Create Vector Indexes.** You may want to create vector indexes on your vector embeddings. This is beneficial for running similarity searches over huge vector spaces.
4. **Query Data with Similarity Searches.** You can then use Oracle AI Vector Search native SQL operations to combine similarity with relational searches to retrieve relevant data.

Optional:

5. **Generate a Prompt and Send it to an LLM for a Full RAG Inference.** You can use the similarity search results to generate a prompt and send it to your generative LLM of choice for a complete RAG pipeline.

Summary



Generate Vector Embeddings

Store Vector Embeddings

Vector Indexes

Query Data with Similarity Searches

In this lesson you have learned about the Oracle AI Vector Search workflow.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Running Basic Queries on Vectors



For Instructor Use Only.

This document should not be distributed.

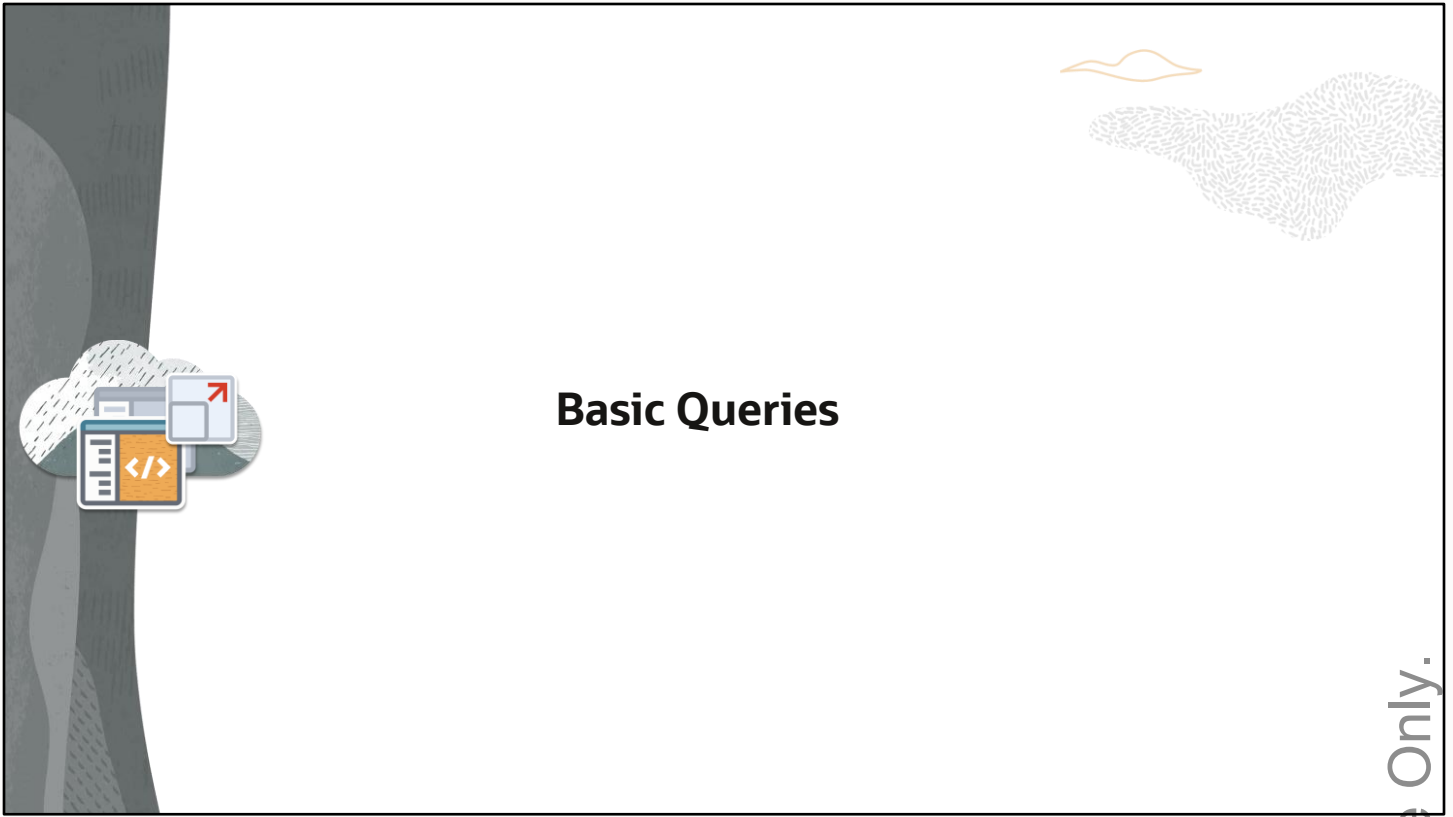
Objectives



Basic Queries

In this lesson, you gain an understanding of running basic queries on vectors.

For Instructor Use Only.
This document should not be distributed.



For Instructor Use Only.
This document should not be distributed.

Basic Queries

- > Queries can contain:
 - Relational data
 - Vector data



Queries can contain a combination of vector data and relational data.

Basic Queries

> Insert data:

```
INSERT INTO t1 VALUES ('[1.1, 2.7, 3.141592653589793238]'),  
                        ('[9.34, 0.0, -6.923]'),  
                        ('[-2.01, 5, -25.8]'),  
                        ('[-8.2, -5, -1013.6]'),  
                        ('[7.3]'),  
                        ('[2.9]'),  
                        ('[1, 2, 3, 4, 5]');
```

The slide shows an example of inserting data into a table.

Basic Queries

> Select data:

```
SELECT * FROM t1;
```

v

```
-----  
[1.10000002E+000,2.70000005E+000,3.14159274E+000]  
[9.34000015E+000,0,-6.92299986E+000]  
[-2.00999999E+000,5.0E+000,-2.57999992E+001]  
[-8.19999981E+000,-5.0E+000,-1.01359998E+003]  
[7.30000019E+000]  
[2.9000001E+000]  
[1.0E+000,2.0E+000,3.0E+000,4.0E+000,5.0E+000]
```

The slide shows an example of running a basic SELECT to retrieve the data.

Basic Queries: Comparison Operations

- > No comparison operations between vectors:

```
SELECT v FROM t1 where v = '[2,9]';
```

```
ERROR at line 1:  
ORA-22848: cannot use VECTOR type as comparison key  
Help: https://docs.oracle.com/error-help/db/ora-22848/
```

```
SELECT v FROM t1 where v <= '[2,9]';
```

```
ERROR at line 1:  
ORA-22848: cannot use VECTOR type as comparison key  
Help: https://docs.oracle.com/error-help/db/ora-22848/
```

It is not possible to perform comparison operations between vectors.

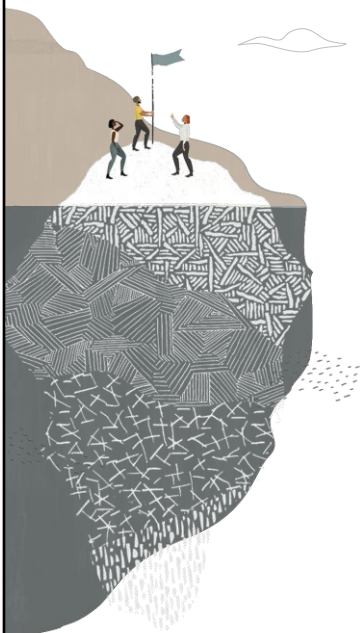


Demo

For Instructor Use Only.

This document should not be distributed.

Summary



Basic Queries

In this lesson, you have learned how to run basic queries on vectors.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Vector Indexes and Memory



For Instructor Use Only.

This document should not be distributed.

Objectives



Vector Indexes

Memory Considerations

In this lesson, you gain an understanding of how to enable the use of vector indexes and also learn about vector memory considerations.

For Instructor Use Only.

This document should not be distributed.



Vector Indexes

For Instructor Use Only.
This document should not be distributed.

Vector Indexes

- > Specialized indexing data structures
- > Use techniques such as:
 - Clustering
 - Partitioning
 - Neighbor graphs
- > Greatly reduces search space
- > Extremely efficient
- > Requires Vector Pool in SGA



You may want to create vector indexes on your vector embeddings and use these indexes for running similarity searches over huge vector spaces.

Vector indexes are a class of specialized indexing data structures that are designed to accelerate similarity searches using high-dimensional vectors. They use techniques such as clustering, partitioning, and neighbor graphs to group vectors representing similar items, which drastically reduces the search space, thereby making the search process extremely efficient.

To allow vector index creation, you must enable a new memory area stored in the SGA called the **Vector Pool**.

Using vector indexing methods can make vector searches faster.

Vector Index Categories

- > Oracle AI Vector Search supports:
 - In-Memory Neighbor Graph vector index
 - Neighbor Partition vector index



In-Memory Neighbor Graph Vector Index

Hierarchical Navigable Small World (HNSW) is the only type of In-Memory Neighbor Graph vector index supported. HNSW graphs are very efficient indexes for vector approximate similarity search. HNSW graphs are structured using principles from small world networks along with layered hierarchical organization.

Neighbor Partition Vector Index

Inverted File Flat (IVF) index is the only type of Neighbor Partition vector index supported. Inverted File Flat Index (IVF Flat or simply IVF) is a partition-based index which balances high search quality with reasonable speed.

Although beyond the scope of this course, some examples of creating vector indexes include:

```
CREATE VECTOR INDEX galaxies_hnsw_idx ON galaxies (embedding) ORGANIZATION INMEMORY  
NEIGHBOR GRAPH
```

```
DISTANCE COSINE
```

```
WITH TARGET ACCURACY 95;
```

```
CREATE VECTOR INDEX galaxies_hnsw_idx ON galaxies (embedding) ORGANIZATION INMEMORY  
NEIGHBOR GRAPH
```

```
DISTANCE COSINE
```

```
WITH TARGET ACCURACY 90 PARAMETERS (type HNSW, neighbors 40, efconstruction 500);
```

```
CREATE VECTOR INDEX galaxies_ivf_idx ON galaxies (embedding) ORGANIZATION NEIGHBOR  
PARTITIONS
```

```
DISTANCE COSINE
```

```
WITH TARGET ACCURACY 95;
```

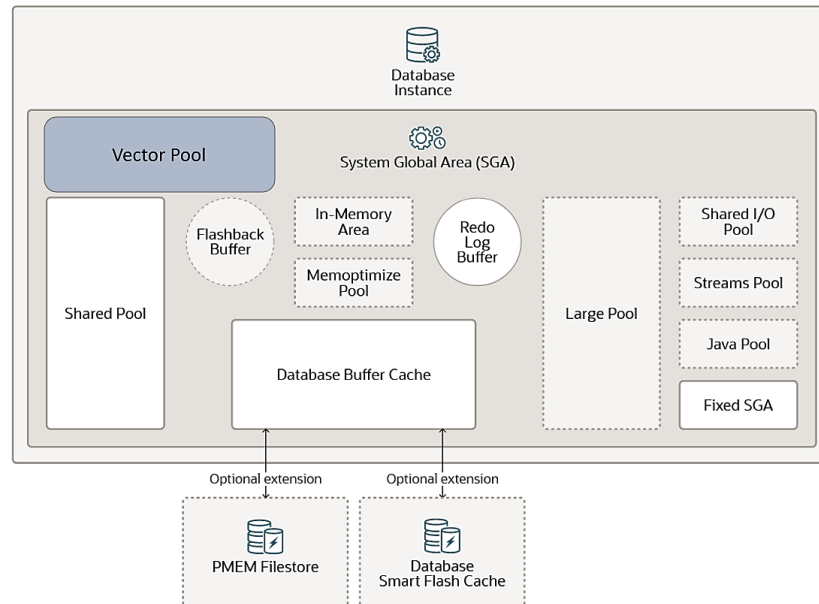
```
CREATE VECTOR INDEX galaxies_ivf_idx ON galaxies (embedding) ORGANIZATION NEIGHBOR  
PARTITIONS
```

```
DISTANCE COSINE
```

```
WITH TARGET ACCURACY 90 PARAMETERS (type IVF, neighbor partitions 10);
```

For Instructor Use Only.
This document should not be distributed.

Vector Pool in the SGA



To allow vector index creation, you must enable a new memory area stored in the SGA called the **Vector Pool**.

The Vector Pool is a memory allocated in SGA to store Hierarchical Navigable Small World (HNSW) vector indexes and all associated metadata. It is also used to speed up Inverted File Flat (IVF) index creation as well as DML operations on base tables with IVF indexes.

Vector Pool

- > The Vector Pool is new to the SGA.
- > VECTOR_MEMORY_SIZE parameter
- > Set at:

- CDB: Current size

```
ALTER SYSTEM SET vector_memory_size=1G SCOPE=BOTH;
```

- PDB: Inherits from CDB
 - Sets maximum usage



To size the Vector Pool, use the **VECTOR_MEMORY_SIZE** initialization parameter. You can dynamically modify this parameter at the following levels:

- At the CDB level **VECTOR_MEMORY_SIZE** specifies the current size of the Vector Pool. Reducing the parameter value will fail if there is current vector usage.
- At the PDB level **VECTOR_MEMORY_SIZE** specifies the maximum Vector Pool usage allowed by a PDB. Reducing the parameter value will be allowed even if current vector usage exceeds the new quota.

You can change the value of a parameter in a parameter file in the following ways:

- By editing an initialization parameter file. In most cases, the new value takes effect the next time you start an instance of the database
- By issuing an ALTER SYSTEM SET ... SCOPE=SPFILE statement to update a server parameter file
- By issuing an ALTER SYSTEM RESET statement to clear an initialization parameter value and set it back to its default value

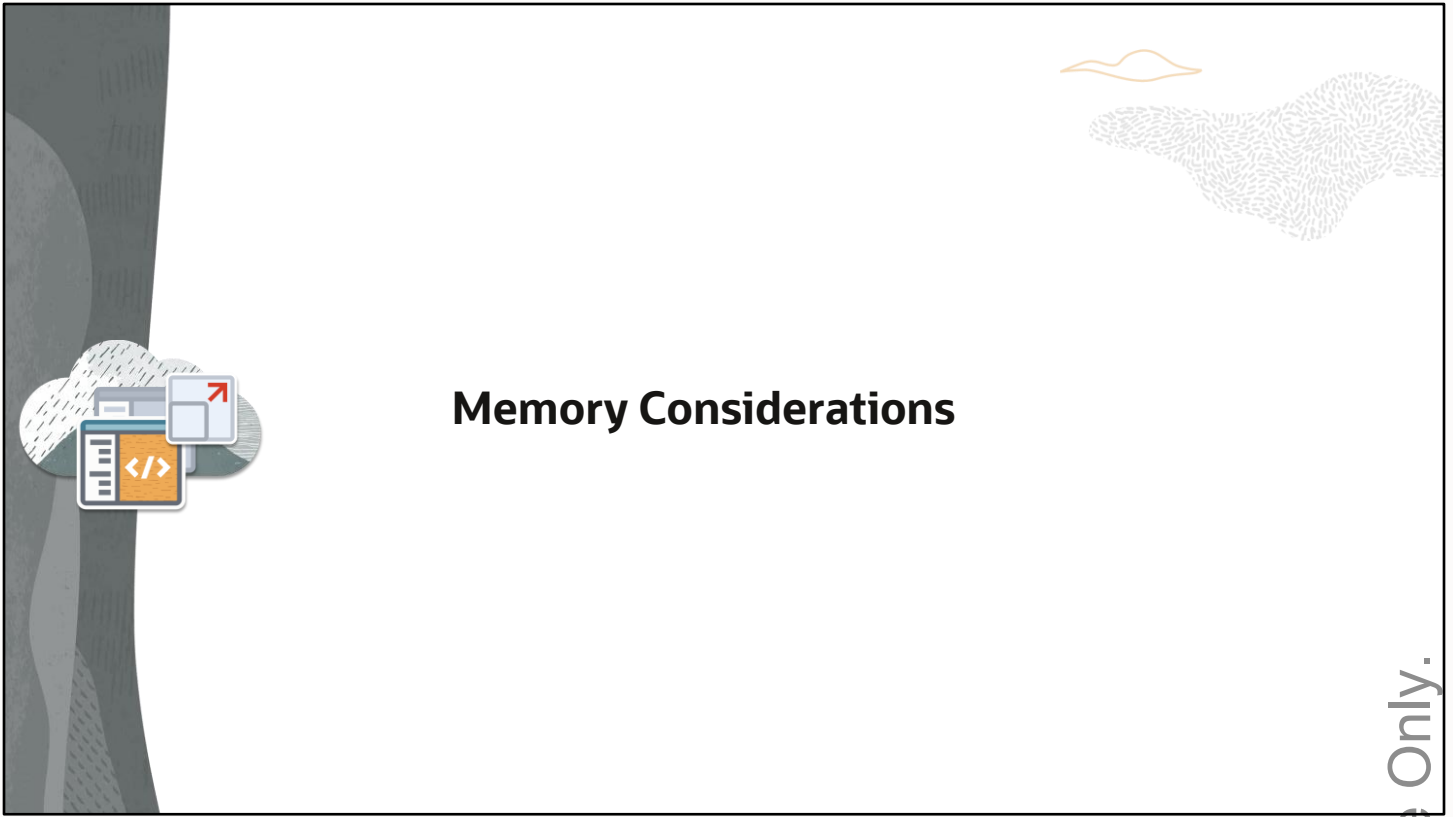
Vector Pool: Other Considerations

- > Vector indexes are stored in this pool.
- > Vector metadata is also stored here.
- > You need to bounce the database.
- > Large vector indexes need lots of RAM.
- > RAM constrains the vector index size.
- > Use IVF indexes when there is not enough RAM.
 - IVF indexes use the buffer cache + disk.



Inverted File Flat (IVF) is a form of Neighbor Partition vector index. It is a partition-based index that achieves search efficiency by narrowing the search area through the use of neighbor partitions or clusters.

For Instructor Use Only.
This document should not be distributed.



Memory Considerations

For Instructor Use Only.
This document should not be distributed.

Memory Considerations

How big is a vector?

- > Most popular vectors are between 1.5 and 12 KB in size.
- > AI Vector Search supports the INT8, FLOAT32, and FLOAT64 formats.

Name	Dimensions	format	Formula	Size Bytes
text-embedding-ada-002	1536	float32	$1536 * 4$	6,144
text-embedding-3-large	3072	float32	$3072 * 4$	12,288
all-MiniLM-L6-v2	384	float32	$384 * 4$	1,536
all-mpnet-base-v2	768	float32	$768 * 4$	3,072
embed-english-v3.0	1024	float32	$1024 * 4$	4,096
embed-english-light-3.0	384	float32	$384 * 4$	1,536
multilingual-e5-large	1024	float32	$1024 * 4$	4,096
multilingual-e5-small	384	float32	$384 * 4$	1,536

What is a vector?

A vector is a numerical representation of text, images, audio, or video that encodes the features/semantic meaning of the data, rather than the actual contents such as the underlying words or pixels. A vector is a list of numerical values, known as dimensions, with a specified format.

How many vector dimensions are supported?

Oracle AI Vector Search supports vectors with up to 65,535 dimensions.

What number formats are supported for vectors?

AI Vector Search supports the INT8, FLOAT32, and FLOAT64 formats.

How big are vectors?

The size of a vector is the product of the number of dimensions and the size of each dimension.

For example, a vector with 2,048 dimensions and the INT8 (1 byte) format is 2 KB in size. A vector with 1,024 dimensions and the FLOAT32 (4 byte) format is 4 KB in size.

How are vectors created?

Vectors are created from different types of input data (text, images, audio, video, etc.) by deep-learning models known as embedding models. You can create vectors outside the database using your own embedding models or embedding service, or you can create them within the database using imported embedding models via the VECTOR_EMBEDDING() SQL function.

Vector Storage

- > Vectors are stored in tables.
- > Table data is stored on disk.

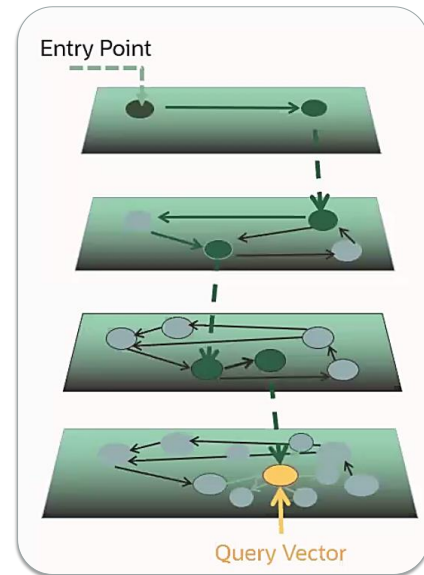
ID	Name	Age	City	Vector
1	Sally	29	San Francisco	[3.1, 0.7, -1.8]
2	Marcus	42	Munich	[2.6, 3.5, -1.2]
3	Anika	25	Chennai	[2.7, 1.6, -0.8]
4	Benjamin	49	New York	[1.2, 7.7, 2.8]
5	Ranjan	31	Mumbai	[4.1, 9.3, 0.2]
6	Xiang	56	Beijing	[0.5, 5.6, 1.5]
7	<u>JaeYeop</u>	28	Seoul	[3.6, 2.7, 0.6]
8	Carlos	33	Monte Video	[2.3, 2.4, -1.2]

For Instructor Use Only.

This document should not be distributed.

Memory Considerations: In-Memory Neighbor Graph Indexes

- > Stored in memory + on disk
- > The in-memory size is:
 - $1.3 * \text{size of vector format}$
 - $* \text{number of dimensions} * \text{number of rows}$
- > This memory is allocated from the SGA.



For Instructor Use Only.
This document should not be distributed.

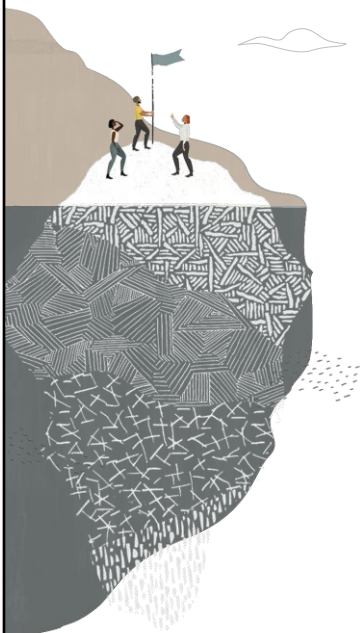
Memory Considerations: Sample Calculation

- > $1.3 * \text{size of vector format} * \text{number of dimensions} * \text{number of rows}$
 - 1.3 is an approximation for the overhead and graph layers.

Constant	Size of vector format	dimensions	rows	GB
1.3	4	384	1,000,000	1.9
1.3	4	768	1,000,000	3.7
1.3	4	1024	1,000,000	5.0
1.3	4	1536	1,000,000	7.4
1.3	4	3072	1,000,000	15.0

For Instructor Use Only.
This document should not be distributed.

Summary



Vector Indexes

Memory Considerations

In this lesson, you have learned how to enable the use of vector indexes and also about the vector memory considerations.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

DML Operations on Vectors



For Instructor Use Only.

This document should not be distributed.

Objectives



Create a Table with a Vector Column

Vector DML

In this lesson, you learn how to add a VECTOR column to a table and perform DML operations.

For Instructor Use Only.
This document should not be distributed.



Create a Table with a Vector Column

For Instructor Use Only.

This document should not be distributed.

VECTOR Data Type

- > New data type to support vector search
- > Definition can include:
 - Number of dimensions
 - Format

```
CREATE TABLE my_vectors (id NUMBER, embedding VECTOR);
```



You don't have to specify the number of dimensions or their format, which are both optional. If you don't specify any of them, you can enter vectors of different dimensions with different formats. This is a simplification to help you get started with using vectors in Oracle Database.

Note: Such vectors typically arise from different embedding models. Vectors from different models (providing a different semantic landscape) are not comparable for use in similarity search.

For Instructor Use Only.
This document should not be distributed.

VECTOR Data Type

- > Another example including:
 - Number of dimensions
 - Format

```
CREATE TABLE my_vectors (id NUMBER, embedding VECTOR(768, INT8)) ;
```



In the given example:

- Each vector that is stored must have 768 dimensions.
- Each dimension will be formatted as an `INT8`.
- The number of dimensions must be strictly greater than zero with no practical upper limit.
- The possible dimension formats are:
 - `INT8` (8-bit integers)
 - `FLOAT32` (32-bit floating-point numbers)
 - `FLOAT64` (64-bit floating-point numbers)
 - `FLOAT32` and `FLOAT64` are IEEE standards. Oracle Database automatically casts the values as needed.

Declaration Formats and Explanation

> VECTOR

- Vectors can have an arbitrary number of dimensions and formats.

> VECTOR(*, *)

- Vectors can have an arbitrary number of dimensions and formats. VECTOR and VECTOR(*,*) are equivalent.

> VECTOR(number_of_dimensions, *) equivalent to VECTOR(number_of_dimensions)

- Vectors must all have the specified number of dimensions or an error is thrown. Every vector will have its dimensions stored without format modification.

> VECTOR(*, dimension_element_format)

- Vectors can have an arbitrary number of dimensions, but their format will be up-converted or down-converted to the specified dimension element format (INT8, FLOAT32, or FLOAT64).

A vector can be NULL but its dimensions cannot (for example, you cannot have a VECTOR with a NULL dimension such as [1.1, NULL, 2.2]).



Vector DML

For Instructor Use Only.
This document should not be distributed.

Vector DML

- > Directly insert vectors into the table:

```
CREATE TABLE galaxies (id NUMBER, name VARCHAR2(50), doc VARCHAR2(500), embedding
VECTOR);

INSERT INTO galaxies VALUES (1, 'M31', 'Messier 31 is a barred spiral galaxy in
the Andromeda constellation which has a lot of barred spiral galaxies.',
'[0,2,2,0,0]');

INSERT INTO galaxies VALUES (9, 'NGC1073', 'NGC 1073 is a barred spiral galaxy in
Cetus constellation.', '[0,1,1,0,0]');
COMMIT;
```

The complete example from the slide, assuming you have already created vectors and know their values:

```
CREATE TABLE galaxies (id NUMBER, name VARCHAR2(50), doc VARCHAR2(500), embedding VECTOR);
INSERT INTO galaxies VALUES (1, 'M31', 'Messier 31 is a barred spiral galaxy in the Andromeda
constellation which has a lot of barred spiral galaxies.', '[0,2,2,0,0]');
INSERT INTO galaxies VALUES (2, 'M33', 'Messier 33 is a spiral galaxy in the Triangulum constellation.',
'[0,0,1,0,0]');
INSERT INTO galaxies VALUES (3, 'M58', 'Messier 58 is an intermediate barred spiral galaxy in the Virgo
constellation.', '[1,1,1,0,0]');
INSERT INTO galaxies VALUES (4, 'M63', 'Messier 63 is a spiral galaxy in the Canes Venatici constellation.',
'[0,0,1,0,0]');
INSERT INTO galaxies VALUES (5, 'M77', 'Messier 77 is a barred spiral galaxy in the Cetus constellation.',
'[0,1,1,0,0]');
INSERT INTO galaxies VALUES (6, 'M91', 'Messier 91 is a barred spiral galaxy in the Coma Berenices
constellation.', '[0,1,1,0,0]');
INSERT INTO galaxies VALUES (7, 'M49', 'Messier 49 is a giant elliptical galaxy in the Virgo constellation.',
'[0,0,0,1,1]');
INSERT INTO galaxies VALUES (8, 'M60', 'Messier 60 is an elliptical galaxy in the Virgo constellation.',
'[0,0,0,0,1]');
INSERT INTO galaxies VALUES (9, 'NGC1073', 'NGC 1073 is a barred spiral galaxy in Cetus constellation.',
'[0,1,1,0,0]');
COMMIT;
```

Vector DML

- > Directly update or delete vectors into the table:

```
UPDATE t2
  SET   v1 = '[2.9]'
 WHERE id = 2;
COMMIT;
```

```
DELETE FROM t2
  WHERE id IN (1, 3);
COMMIT;
```

The examples on the slide assume you have already created vectors and know their values.

For Instructor Use Only.
This document should not be distributed.

Vector DML Using SQL*Loader

- > You can also load data using SQL*Loader:

```
host sqlldr vector/vector@CDB1_PDB1 control=galaxies_vec.ctl log=galaxies_vec.log
SQL*Loader: Release 23.0.0.0.0 - Development on Thu Jan 11 19:46:21 2024 Version
23.4.0.23.00 Copyright (c) 1982, 2024, Oracle and/or its affiliates. All rights
reserved.
```

```
Path used: Conventional Commit point reached - logical record count 10
```

```
Table GALAXIES2: 9 Rows successfully loaded.
```

```
Check the log file: galaxies_vec.log for more information about the load.
```

In the example on the slide, you can see how to create a .csv file and you can use SQL*Loader to load vector data into a five-dimension vector space.

Let's imagine we have the following text documents classifying galaxies by their types:

DOC1: "Messier 31 is a barred spiral galaxy in the Andromeda constellation which has a lot of barred spiral galaxies."

DOC2: "Messier 33 is a spiral galaxy in the Triangulum constellation."

DOC3: "Messier 58 is an intermediate barred spiral galaxy in the Virgo constellation."

DOC4: "Messier 63 is a spiral galaxy in the Canes Venatici constellation."

DOC5: "Messier 77 is a barred spiral galaxy in the Cetus constellation."

DOC6: "Messier 91 is a barred spiral galaxy in the Coma Berenices constellation."

DOC7: "NGC 1073 is a barred spiral galaxy in the Cetus constellation."

DOC8: "Messier 49 is a giant elliptical galaxy in the Virgo constellation."

DOC9: "Messier 60 is an elliptical galaxy in the Virgo constellation."

You can create vectors representing the preceding galaxy's classes using the following five-dimension vector space based on the count of important words appearing in each document:

Five dimension vector space table:

Galaxy Classes	Intermediate Elliptical	Barred	Spiral	Giant
M31	0 0	2	2	0
M33	0 0	0	1	0
M58	1 0	1	1	0
M63	0 0	0	1	0
M77	0 0	1	1	0
M91	0 0	1	1	0
M49	0 1	0	0	1
M60	0 1	0	0	0
NGC1073	0 0	1	1	0

This naturally gives you the following vectors:

M31: [0,2,2,0,0]
M33: [0,0,1,0,0]
M58: [1,1,1,0,0]
M63: [0,0,1,0,0]
M77: [0,1,1,0,0]
M91: [0,1,1,0,0]
M49: [0,0,0,1,1]
M60: [0,0,0,0,1]
NGC1073: [0,1,1,0,0]

For Instructor Use Only.
This document should not be distributed.

Based on the data described previously, you can create the following galaxies_vec.csv file:

1:M31:Messier 31 is a barred spiral galaxy in the Andromeda constellation which has a lot of barred spiral galaxies.: [0,2,2,0,0]:

2:M33:Messier 33 is a spiral galaxy in the Triangulum constellation.: [0,0,1,0,0]:

3:M58:Messier 58 is an intermediate barred spiral galaxy in the Virgo constellation.: [1,1,1,0,0]:

4:M63:Messier 63 is a spiral galaxy in the Canes Venatici constellation.: [0,0,1,0,0]:

5:M77:Messier 77 is a barred spiral galaxy in the Cetus constellation.: [0,1,1,0,0]:

6:M91:Messier 91 is a barred spiral galaxy in the Coma Berenices constellation.: [0,1,1,0,0]:

7:M49:Messier 49 is a giant elliptical galaxy in the Virgo constellation.: [0,0,0,1,1]:

8:M60:Messier 60 is an elliptical galaxy in the Virgo constellation.: [0,0,0,0,1]:

9:NGC1073:NGC 1073 is a barred spiral galaxy in the Cetus constellation.: [0,1,1,0,0]:

An example control file referenced above could look like this:

```
recoverable
LOAD DATA
infile 'galaxies_vec.csv'
INTO TABLE galaxies
fields terminated by ':'
trailing nullcols
(
id,
name char (50),
doc char (500),
embedding char (32000)
)
```

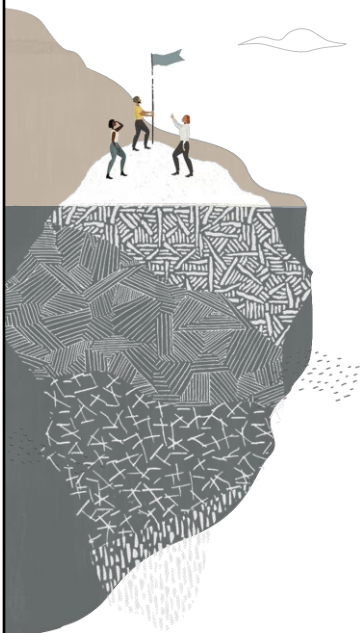
For Instructor Use Only.
This document should not be distributed.



Demo

For Instructor Use Only.
This document should not be distributed.

Summary



Create a Table with a Vector Column

Vector DML

In this lesson, you have gained an understanding of how to add a VECTOR column to a table and perform DML operations.

For Instructor Use Only.

This document should not be distributed.

Vector DDL



For Instructor Use Only.
This document should not be distributed.

Objectives



Tables with Different Vector Formats

DDL Operations on Vectors

Prohibited Operations

In this lesson, you gain an understanding of working with tables with different vector formats, performing DDL on vectors, and also prohibited operations.



Tables with Different Vector Formats

For Instructor Use Only.
This document should not be distributed.

Tables with Different Vector Formats

- > Can have:
 - More than one column of VECTOR data type
 - Vector columns with different formats



It is possible to have more than one column that is a VECTOR data type. You may also have different formats for the vector columns in one table.

For Instructor Use Only.
This document should not be distributed.

Tables with Different Vector Formats: Example

- > Can have:
 - More than one column of VECTOR data type
 - Vector columns with different formats

```
CREATE TABLE IF NOT EXISTS t5
( v1      VECTOR(3, float32),
  v2      VECTOR(2, float64),
  v3      VECTOR(1, int8),
  v4      VECTOR(1, *),
  v5      VECTOR(*, float32),
  v6      VECTOR(*, *),
  v7      VECTOR
);
```

For Instructor Use Only.
This document should not be distributed.

Tables with Different Vector Formats: Insert Example

- > Insert one row with different formats:

```
INSERT INTO t5 VALUES ('[1.1, 2.2, 3.3]',  
                        '[1.1, 2.2]',  
                        '[7]',  
                        '[9]',  
                        '[1.1, 2.2, 3.3, 4.4, 5.5]',  
                        '[1.1, 2.2]',  
                        '[1.1, 2.2, 3.3, 4.4, 5.5, 6.6]'  
                        );
```

In the example on the slide shows how to insert a row with different formats into the t5 table.

For Instructor Use Only.
This document should not be distributed.



DDL Operations on Vectors



For Instructor Use Only.
This document should not be distributed.

DDL Operations on Vectors

- > Add vector columns to existing tables:

```
ALTER TABLE t6 ADD v2 VECTOR(2, float32);
```



The example in the slide shows how to add a column called v2 to the t6 table.

For Instructor Use Only.
This document should not be distributed.

DDL Operations on Vectors

- > Drop vector columns from existing tables:

```
ALTER TABLE t6 DROP COLUMN v2;
```

The example on the slide shows how to drop a column called v2 from the t6 table. You may also drop the entire table instead if required, depending on your privileges.



Prohibited Operations

For Instructor Use Only.

This document should not be distributed.

VECTOR Data Type Restrictions

- > You cannot define VECTOR columns in/as:
 - External Tables
 - IOTs (neither as Primary Key nor as non-Key column)
 - Clusters/Cluster Tables
 - Global Temp Tables
 - (Sub)Partitioning Key
 - Primary Key
 - Foreign Key
 - Unique Constraint



The slide depicts some of the VECTOR data type restrictions.

For Instructor Use Only.
This document should not be distributed.

VECTOR Data Type Restrictions

- > You cannot define VECTOR columns in/as:
 - Check Constraint
 - Default Value
 - Modify Column
 - MSSM tablespace (only SYS user can create VECTORs as Basicfiles in MSSM tablespace)
 - CQN queries
 - Non-vector indexes such as B-tree, Bitmap, Reverse Key, Text, Spatial Indexes, etc.



Oracle Database does not support the following SQL constructs with VECTOR columns:

- Distinct, Count Distinct
- Order By, Group By
- Join condition
- Comparison operators (e.g. >, <, =), etc.

For Instructor Use Only.
This document should not be distributed.



Demo

For Instructor Use Only.

This document should not be distributed.

Summary



Tables with Different Vector Formats

DDL Operations on Vectors

Prohibited Operations

In this lesson, you gained an understanding of working with tables with different vector formats, performing DDL on vectors, and also prohibited operations.

For Instructor Use Only.

This document should not be distributed.

Creating and Finding the Nearest Vectors



For Instructor Use Only.

This document should not be distributed.

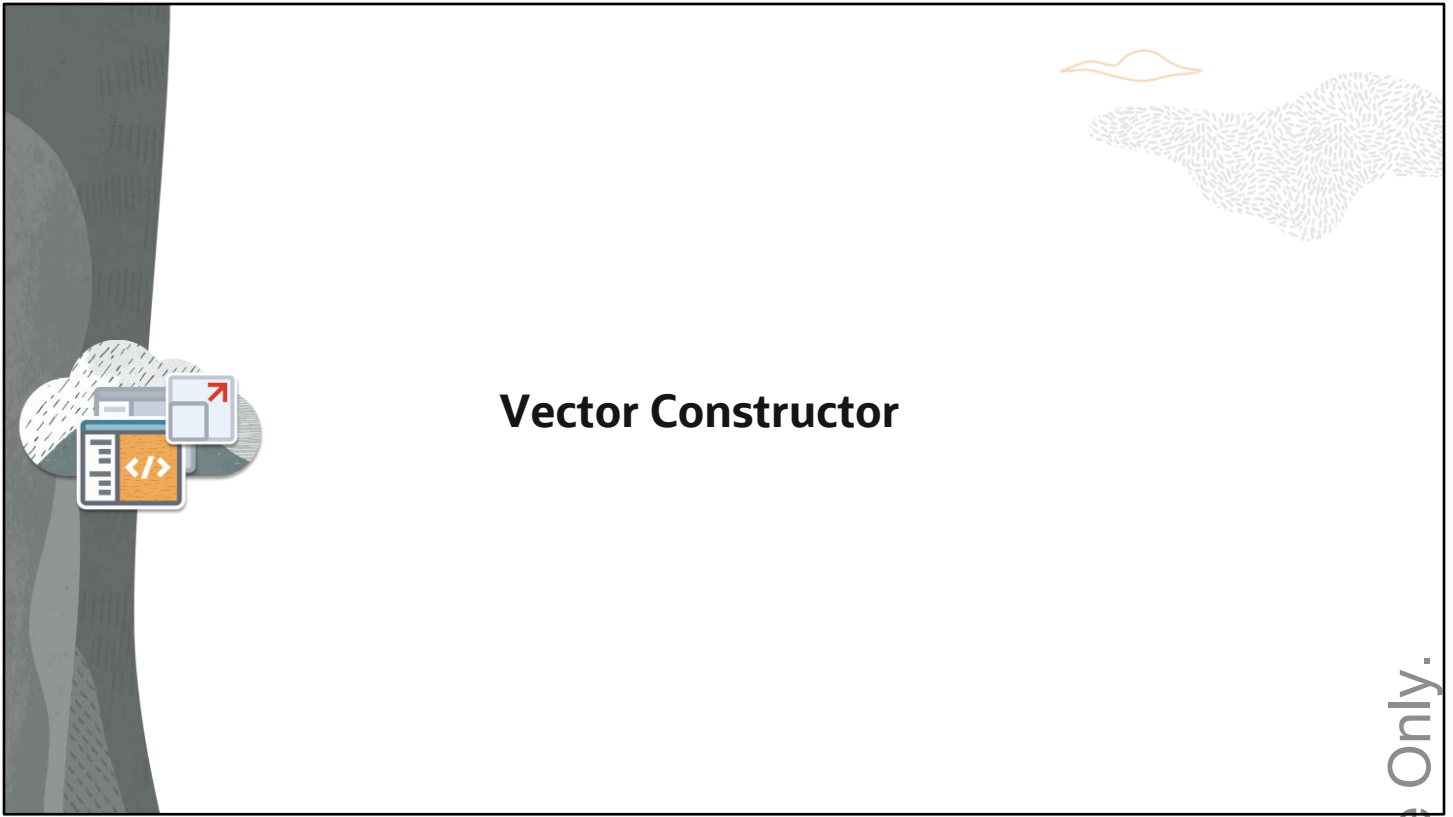
Objectives



Vector Constructor

Vector Distance

In this lesson, you gain an understanding of the vector constructor and the vector distance functions.



Vector Constructor

For Instructor Use Only.
This document should not be distributed.

Vector Constructor

- > Create vectors without storing them.
- > Useful for learning purposes
 - Use with a small number of dimensions.
- > Specify vector values.
 - Dimensions optional
 - Format optional



There are a number of SQL functions and operators that you can use with vectors in Oracle AI Vector Search. One such operator is the Vector Constructor.

You can create vectors without inserting them into a vector column. This is useful for learning how to use vectors with a small number of dimensions. Vectors created via Embedding Models from companies like OpenAI or Cohere can have thousands of dimensions.

You need to specify the vector values, with optional numbers of dimensions and format.

For Instructor Use Only.

This document should not be distributed.

Vector Constructor Examples

```
SELECT VECTOR(' [0,0] ');
```

```
VECTOR(' [0,0] ')  
-----  
[0,0]
```

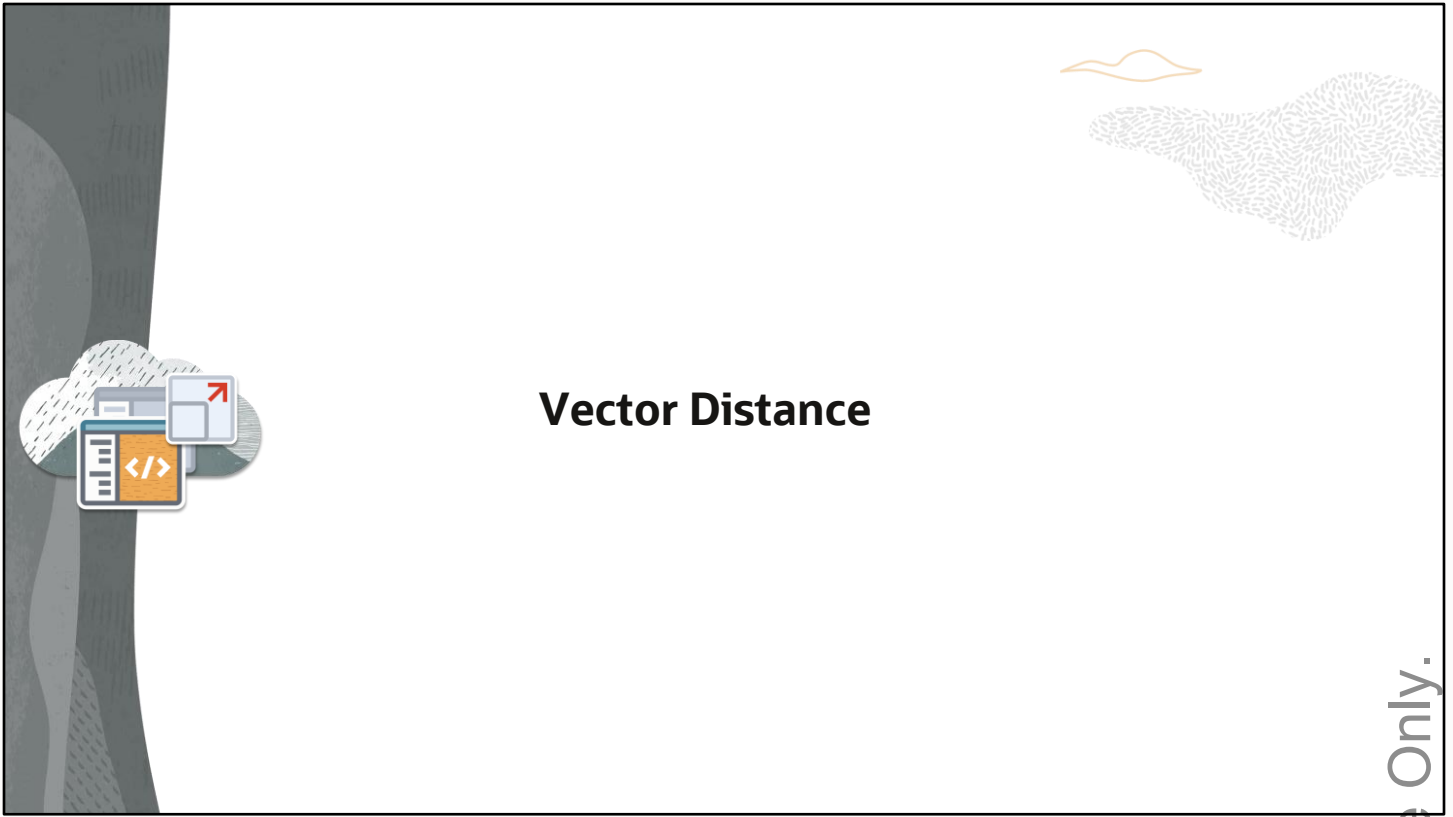
```
SELECT VECTOR(' [10,0] ');
```

```
VECTOR(' [10,0] ')  
-----  
[1.0E+001,0]
```

```
SELECT VECTOR(' [0,5] ', 2, float32);
```

```
VECTOR(' [0,5] ',2,FLOAT32)  
-----  
[0,5.0E+000]
```

The example constructors in the slide represent two-dimensional coordinates. In Oracle 23ai, you do not need to specify a table for a SQL function. The values in the vector will be in a scientific format.



For Instructor Use Only.
This document should not be distributed.

Vector Distance Operand

- > VECTOR_DISTANCE() is the main function.
- > Takes two vectors as parameters
- > Metric is optional.
 - Default is COSINE.



VECTOR_DISTANCE() is the main function that allows you to calculate distances between two vectors.

The VECTOR_DISTANCE() function takes two vectors as parameters. You can optionally specify a distance metric to calculate the distance desirably. If you do not specify a distance metric, then the default distance metric is the COSINE metric.

You can optionally use the following shorthand functions too: L1_DISTANCE, L2_DISTANCE, COSINE_DISTANCE, and INNER_PRODUCT. These functions take two vectors as input and return the distance between them.

All of these functions return the vectors' distance as a BINARY_DOUBLE.

When using the VECTOR_DISTANCE() function to perform a similarity search, note the following caveats:

- If a similarity search query *does not* specify a distance metric in the VECTOR_DISTANCE() function, then the default COSINE metric will be used for both exact and approximate (index-based) searches.
- If a similarity search query does specify a distance metric in the VECTOR_DISTANCE() function, then an exact search with that distance metric is used if it conflicts with the distance metric specified in a vector index. If the two distance metrics are the same, then that will be used for both exact and approximate (index-based) searches.

Vector Distance Metrics

- > Euclidean and Euclidean Squared Distances
- > Cosine Similarity
- > Dot Product Similarity
- > Manhattan Distance
- > Hamming Similarity



A vector distance function takes in two vector operands and a distance metric to compute a mathematical distance between those two vectors based on the distance metric provided. Distances determine similarity or dissimilarity between vectors.

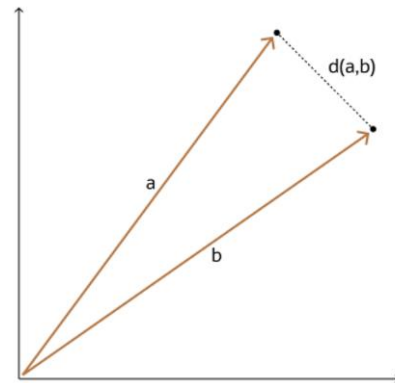
Vector Distance Metrics

Measuring distances in a vector space is at the heart of identifying the most relevant results for a given query vector. That process is very different from the well-known keyword filtering in the relational database world.

When working with vectors, there are several ways you can calculate distances to determine how similar, or dissimilar, two vectors are. Each distance metric is computed using different mathematical formulas. The time it takes to calculate the distance between two vectors depends on many factors, including the distance metric used as well as the format of the vectors themselves, such as the number of vector dimensions and the vector dimension formats. Generally it's best to match the distance metric you use to the one that was used to train the vector embedding model that generated the vectors. We will look at the listed metrics over the next few pages.

Euclidean and Euclidean Squared Distances

- > Straight-line distance between two vectors
- > Uses the Pythagorean theorem
- > Sensitive to vector:
 - Size
 - Direction



[Euclidean and Euclidean Squared Distances \(https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/euclidean-and-squared-euclidean-distances.html\)](https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/euclidean-and-squared-euclidean-distances.html)

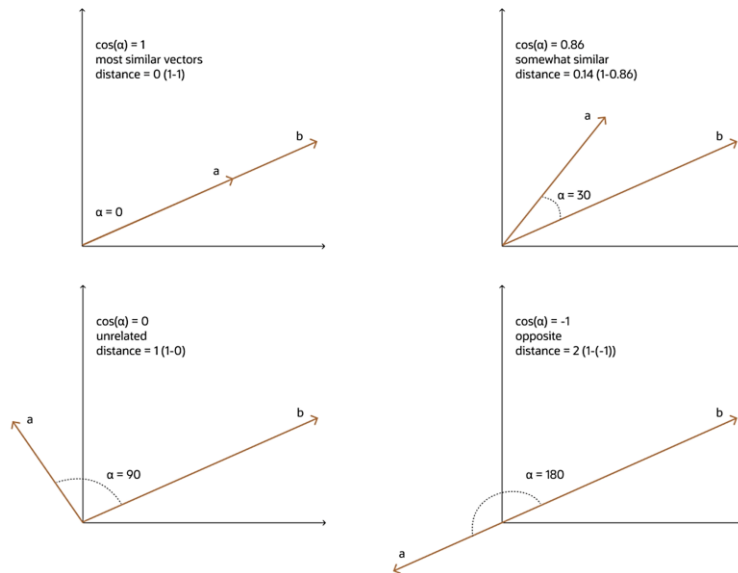
Euclidean distance reflects the distance between each of the vectors' coordinates being compared – basically the straight-line distance between two vectors. This is calculated using the Pythagorean theorem applied to the vector's coordinates ($\text{SQRT}(\text{SUM}((x_i - y_i)^2))$).

This metric is sensitive to both the vector's size and its direction.

With Euclidean distances, comparing squared distances is equivalent to comparing distances. So, when ordering is more important than the distance values themselves, the Squared Euclidean distance is very useful as it is faster to calculate than the Euclidean distance (avoiding the square-root calculation).

Cosine Similarity

- > Widely used
- > Smaller the angle
 - More similar



[Cosine Similarity](https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/cosine-similarity.html) (<https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/cosine-similarity.html>)

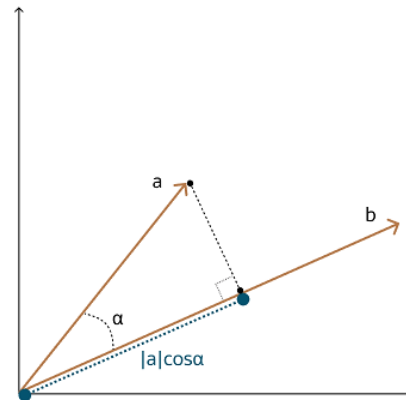
One of the most widely used similarity metric, especially in natural language processing (NLP), is cosine similarity, which measures the cosine of the angle between two vectors.

The smaller the angle, the more similar are the two vectors. Cosine similarity measures the similarity in the direction or angle of the vectors, ignoring differences in their size (also called *magnitude*). The smaller the angle, the bigger is its cosine. So the cosine distance and the cosine similarity have an inverse relationship. While cosine distance measures how different two vectors are, cosine similarity measures how similar two vectors are.

This example shows four graphs that depict cosine similarity. Each graph has two vectors a and b , and measures the cosine of the angle between these two vectors.

Dot Product Similarity

- > Multiply size of each vector by the cosine of their angle.
- > Equivalent to sum of vector's coordinates
- > Larger means more similar.
- > Smaller means less similar.



[Dot Product Similarity \(https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/dot-product-similarity.html\)](https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/dot-product-similarity.html)

The dot product similarity of two vectors can be viewed as multiplying the size of each vector by the cosine of their angle. The corresponding geometrical interpretation of this definition is equivalent to multiplying the size of one of the vectors by the size of the projection of the second vector onto the first one, or vice versa.

You project one vector on the other and multiply the resulting vector sizes.

Incidentally, this is equivalent to the sum of the products of each vector's coordinate. Often, you do not have access to the cosine of the two vector's angle, hence this calculation is easier.

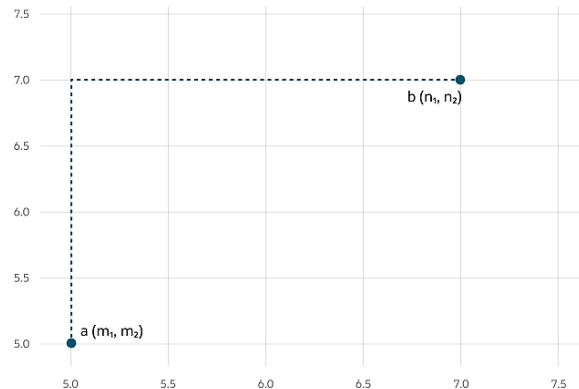
Larger dot product values imply that the vectors are more similar, while smaller values imply that they are less similar. Compared to using Euclidean distance, using the dot product similarity is especially useful for high-dimensional vectors.

Note that normalizing vectors and using the dot product similarity is equivalent to using cosine similarity. There are cases where dot product similarity is faster to evaluate than cosine similarity, and conversely where cosine similarity is faster than dot product similarity. A normalized vector is created by dividing each dimension by the norm (or length) of the vector, such that the norm of the normalized vector is equal to 1.

The example on the slide shows a graph that depicts dot product similarity. The graph has two vectors a and b that projects one vector on the other and multiplies the resulting vector sizes.

Manhattan Distance

- > Useful for describing uniform grids:
 - City blocks
 - Power grids
 - Chessboard
- > Faster than Euclidean metric



[Manhattan Distance](https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/manhattan-distance.html) (<https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/manhattan-distance.html>)

This metric is calculated by summing the distance between the dimensions of the two vectors that you want to compare.

Imagine yourself in the streets of Manhattan trying to go from point A to point B. A straight line is not possible.

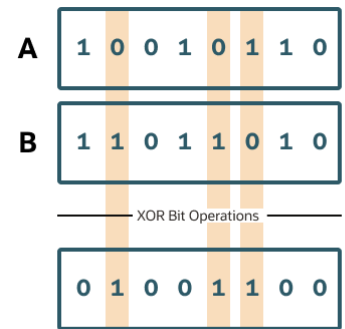
This metric is most useful for vectors describing objects on a uniform grid, such as city blocks, power grids, or a chessboard. It can be useful for higher dimensional vector spaces too. Compared to the Euclidean metric, the Manhattan metric is faster for calculations and you can use it advantageously for higher dimensional vector spaces.

For Instructor Use Only.

This document should not be distributed.

Hamming Similarity

- > Describes where vector dimensions differ
- > Binary vectors:
 - Number of bits require change to match
- > Compares the position of each bit in the sequence
- > Used for network error detection



Hamming Distance = 3

[Hamming Similarity \(https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/hamming-distance.html\)](https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/hamming-distance.html)

The Hamming distance between two vectors represents the number of dimensions where they differ.

For example, when using binary vectors, the Hamming distance between two vectors is the number of bits you must change to change one vector into the other. To compute the Hamming distance between two vectors, you need to compare the position of each bit in the sequence. You can do this by using `exclusive or` (also called the XOR bit operation), which outputs 1 if the bits in the sequence do not match, and 0 otherwise. It's important to note that the bit strings need to be of equal length for the comparison to make sense.

The Hamming metric is mainly used with binary vectors for error detection over networks.

For Instructor Use Only.

This document should not be distributed.

Vector Distance Examples

```
SELECT TO_NUMBER(VECTOR_DISTANCE(  
  VECTOR('[0, 0]'),  
  VECTOR('[10, 0]'),  
  EUCLIDEAN)) DISTANCE;
```

```
DISTANCE  
-----  
10
```

```
SELECT TO_NUMBER(VECTOR_DISTANCE(  
  VECTOR('[3.14, 2.718]'),  
  VECTOR('[-5.3, 4.2]'), EUCLIDEAN)) DISTANCE;
```

```
DISTANCE  
-----  
8.56912708
```

The two examples on the slide depicts the use of the `vector_distance()` SQL function with Euclidean metric to return the distance between two vectors.

For Instructor Use Only.
This document should not be distributed.

Shorthand Operators For Distances

> **<-> is the Euclidian distance operator.**

- Equivalent to L2_DISTANCE(expr1, expr2) or VECTOR_DISTANCE(expr1, expr2, EUCLIDEAN)

> **<=> is the cosine distance operator.**

- Equivalent to COSINE_DISTANCE(expr1, expr2) or VECTOR_DISTANCE(expr1, expr2, COSINE)

> **<#> is the negative dot product operator.**

- Equivalent to -1*INNER_PRODUCT(expr1, expr2) or VECTOR_DISTANCE(expr1, expr2, DOT)

```
'[1, 2]' <-> '[0,1]'
```

Oracle AI Vector Search provides shorthand operators that you can use for distance functions.

Oracle provides the following shorthand distance operators in lieu of their corresponding distance functions:

- **<-> is the Euclidian distance operator:** `expr1 <-> expr2` is equivalent to `L2_DISTANCE(expr1, expr2)` or `VECTOR_DISTANCE(expr1, expr2, EUCLIDEAN)`
- **<=> is the cosine distance operator:** `expr1 <=> expr2` is equivalent to `COSINE_DISTANCE(expr1, expr2)` or `VECTOR_DISTANCE(expr1, expr2, COSINE)`
- **<#> is the negative dot product operator:** `expr1 <#> expr2` is equivalent to `-1*INNER_PRODUCT(expr1, expr2)` or `VECTOR_DISTANCE(expr1, expr2, DOT)`

Syntax

`<expr1> <-> <expr2>`

`<expr1> <#> <expr2>`

`<expr1> <=> <expr2>`

Parameters

`expr1` and `expr2` must evaluate to vectors and have the same number of dimensions. These operations return NULL if either `expr1` or `expr2` is NULL.

Example

- '[1, 2]' <-> '[0,1]'
- $v1 <-> '[' || '1,2,3' || ']'$ is equivalent to $v1 <-> '[1, 2, 3]'$
- $v1 <-> '[1,2]'$ is equivalent to $L2_DISTANCE(v1, '[1,2]')$
- $v1 <=> v2$ is equivalent to $COSINE_DISTANCE(v1, v2)$
- $v1 <\#> v2$ is equivalent to $-1*INNER_PRODUCT(v1, v2)$

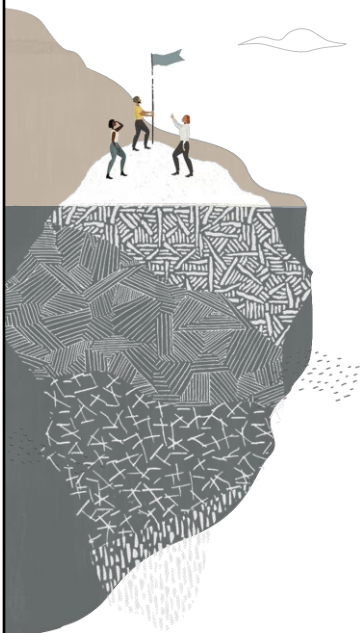
For Instructor Use Only.
This document should not be distributed.



Demo

For Instructor Use Only.
This document should not be distributed.

Summary



Vector Constructor

Vector Distance

In this lesson, you learned about the vector constructor and the vector distance functions.

For Instructor Use Only.
This document should not be distributed.

Finding the Closest Vectors



For Instructor Use Only.

This document should not be distributed.

Objectives



Exact Similarity Search

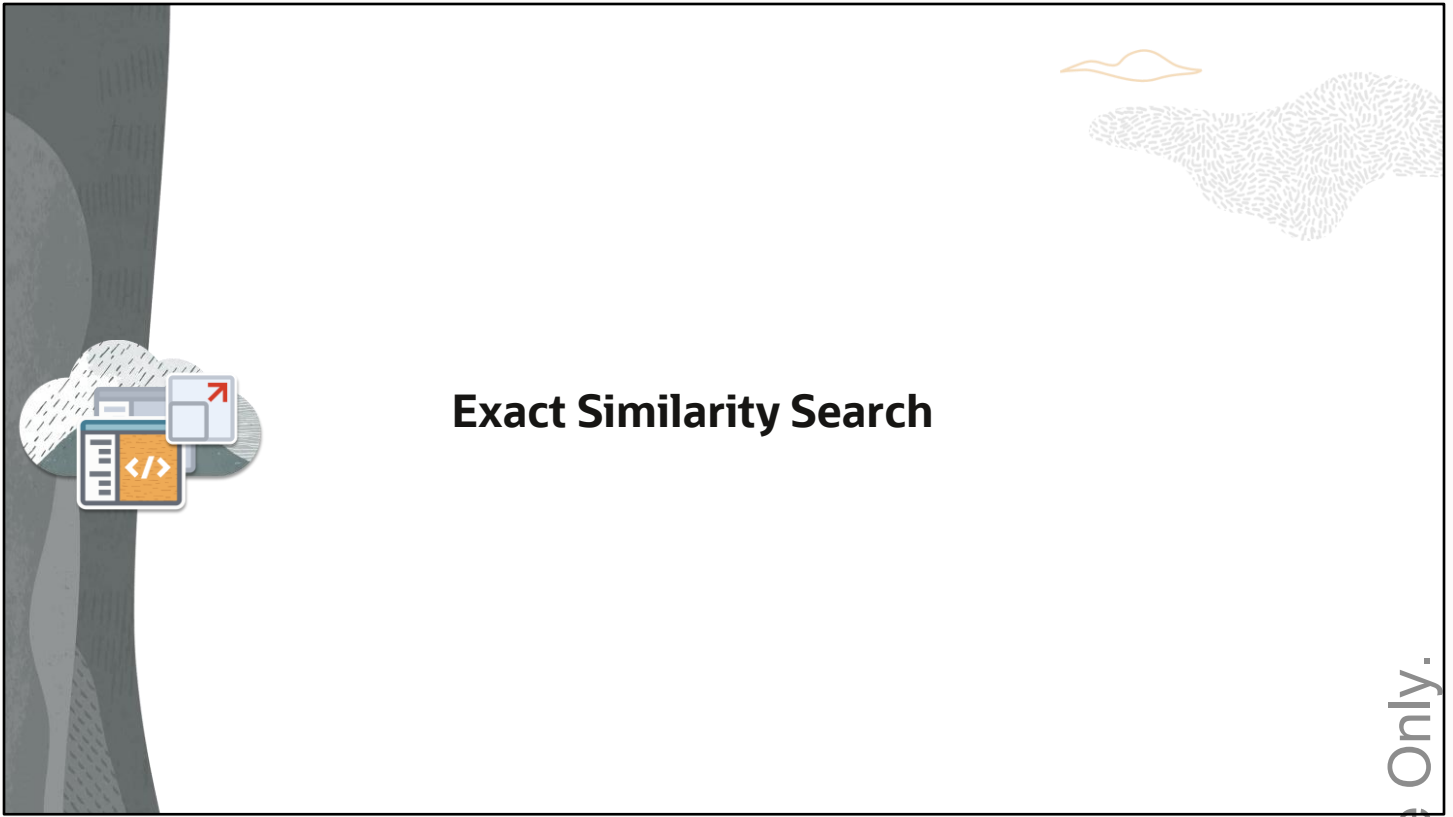
Approximate Similarity Search

Multi-Vector Similarity Search

In this lesson, you learn about running basic queries on vectors including:

- Exact Similarity Search
- Approximate Similarity Search
- Multi-Vector Similarity Search

For Instructor Use Only.
This document should not be distributed.



Exact Similarity Search

For Instructor Use Only.

This document should not be distributed.

Exact Similarity Search

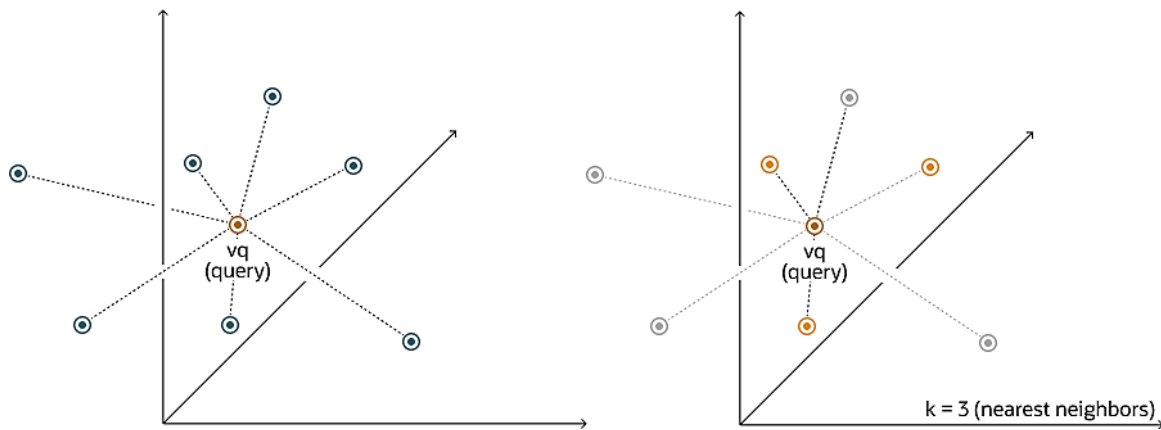
- > Calculates query vector distance to all other vectors
- > Also called flat search
- > Most accurate results
- > Perfect search quality
- > But involves potentially significant search times



A similarity search looks for the relative order of vectors compared to a query vector. Naturally, the comparison is done using a particular distance metric but what is important is the result set of your top closest vectors, not the distance between them.

For Instructor Use Only.
This document should not be distributed.

Exact Similarity Search



As an example, and given a certain query vector, you can calculate its distance to all other vectors in your data set. This type of search, also called flat search, or exact search, produces the most accurate results with perfect search quality. However, this comes at the cost of significant search times.

With an exact search, you compare the query vector v_q against every other vector in your space by calculating its distance to each vector. After calculating all of these distances, the search returns the nearest k of those as the nearest matches. This is called a k -nearest neighbors (kNN) search.

Euclidean

```
SELECT docID
FROM vector_tab
ORDER BY VECTOR_DISTANCE( embedding, :query_vector, EUCLIDEAN )
FETCH EXACT FIRST 10 ROWS ONLY;
```

For example, the Euclidean similarity search involves retrieving the top-k nearest vectors in your space relative to the Euclidean distance metric and a query vector.

The example on the slide retrieves the top 10 vectors from the `vector_tab` table that are the nearest to `query_vector`. The `docID` and `embedding` are columns defined in the `vector_tab` table and `embedding` has the `VECTOR` data type.

Euclidean Squared Distance

```
SELECT docID
FROM vector_tab
ORDER BY VECTOR_DISTANCE( embedding, :query_vector)
FETCH FIRST 10 ROWS ONLY;
```

In the case of Euclidean distances, comparing squared distances is equivalent to comparing distances. So, when ordering is more important than the distance values themselves, the Euclidean Squared distance is very useful as it is faster to calculate than the Euclidean distance (avoiding the square-root calculation). Consequently, it is simpler and faster to rewrite the query, as in the example above.

Euclidean Squared distance is the Oracle AI Vector Search default when using the `VECTOR_DISTANCE()` function and no vector indexes are created with a different distance metric on the relevant vector columns. **Note that the `EXACT` keyword is optional.**

Note: Ensure that you use the distance function that was used to train your embedding model.



Approximate Similarity Search

For Instructor Use Only.

This document should not be distributed.

Approximate Similarity Search

- > Uses vector indexes
- > Can be more efficient
- > Can be less accurate
- > Uses target accuracy



For a vector search to be useful, it needs to be fast and accurate. Approximate similarity searches seek a balance between these goals.

For faster search speeds with large vector spaces, you can use approximate similarity search using vector indexes.

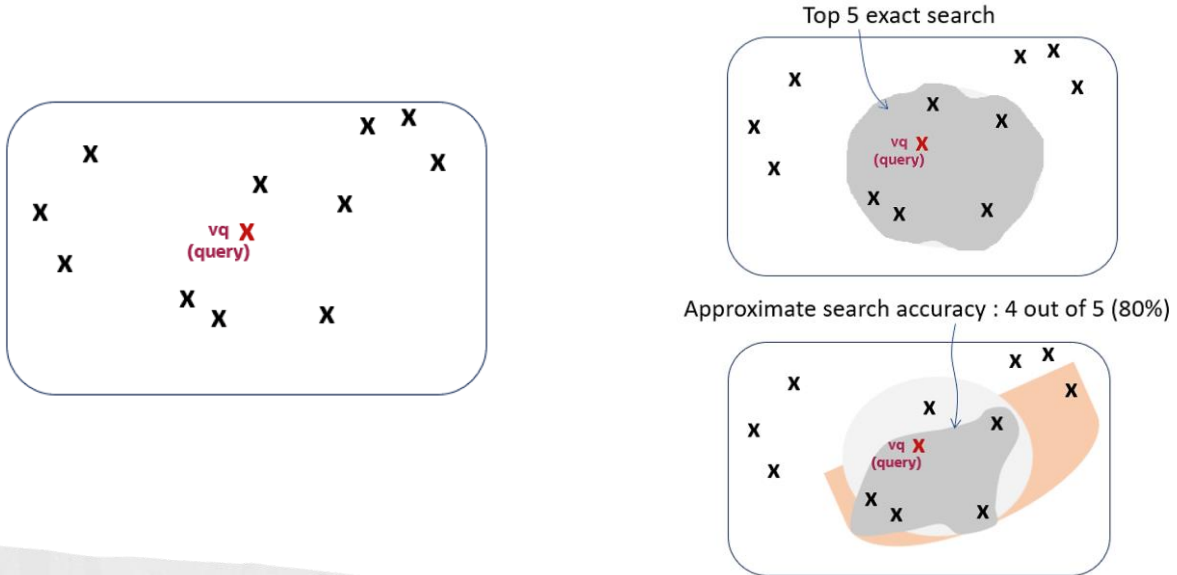
Using vector indexes for a similarity search is called an **approximate search**. Approximate searches use **vector indexes**, which trade off accuracy for performance.

There are many types of approximate searches that you can perform using vector indexes. Vector indexes can be less accurate, but they can consume less resources, and can be more efficient. Unlike traditional database indexes, vector indexes are constructed and perform searches using heuristic-based algorithms.

Because 100 percent accuracy cannot be guaranteed by heuristics, vector index searches use **target accuracy**. Internally, the algorithms used for both index creation and index search are doing their best to be as accurate as possible. However, you have the possibility to influence those algorithms by specifying a target accuracy.

When creating the index or searching it, you can specify non-default target accuracy values either by specifying a percentage value, or by specifying internal parameters values, depending on the index type you are using.

Approximate Similarity Search or Exact Similarity Search?



To better understand what is meant by target accuracy look at the examples above.

The first example illustrates a vector space where each vector is represented by a small cross. The one in red represents your query vector.

Running a top-5 exact similarity search in that context would return the five vectors shown in the second example.

Compared to the exact search, the similarity search retrieved four out of five vectors correctly. The similarity search has 80% accuracy compared to the exact similarity search. This is illustrated in the third example.

Approximate Similarity Search

- > Uses vector indexes
 - Hierarchical Navigable Small World (HNSW) index
 - Inverted File Flat (IVF) index



Approximate similarity search uses vector indexes.

For Instructor Use Only.
This document should not be distributed.

Approximate Similarity Search: HNSW

- > Hierarchical Navigable Small World (HNSW) index

```
CREATE VECTOR INDEX galaxies_hnsw_idx ON galaxies (embedding) ORGANIZATION INMEMORY  
NEIGHBOR GRAPH DISTANCE COSINE WITH TARGET ACCURACY 95;  
  
SELECT name FROM galaxies WHERE name <> 'NGC1073' ORDER BY VECTOR_DISTANCE(  
embedding, to_vector('[0,1,1,0,0]'), COSINE ) FETCH APPROXIMATE FIRST 3 ROWS ONLY;
```

[Approximate Search Using HNSW \(https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/approximate-search-using-hnsw.html\)](https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/approximate-search-using-hnsw.html)

The example on the slide shows how you can create the Hierarchical Navigable Small World (HNSW) index and run an approximate search using that index.

For Instructor Use Only.
This document should not be distributed.

Approximate Similarity Search: IVF

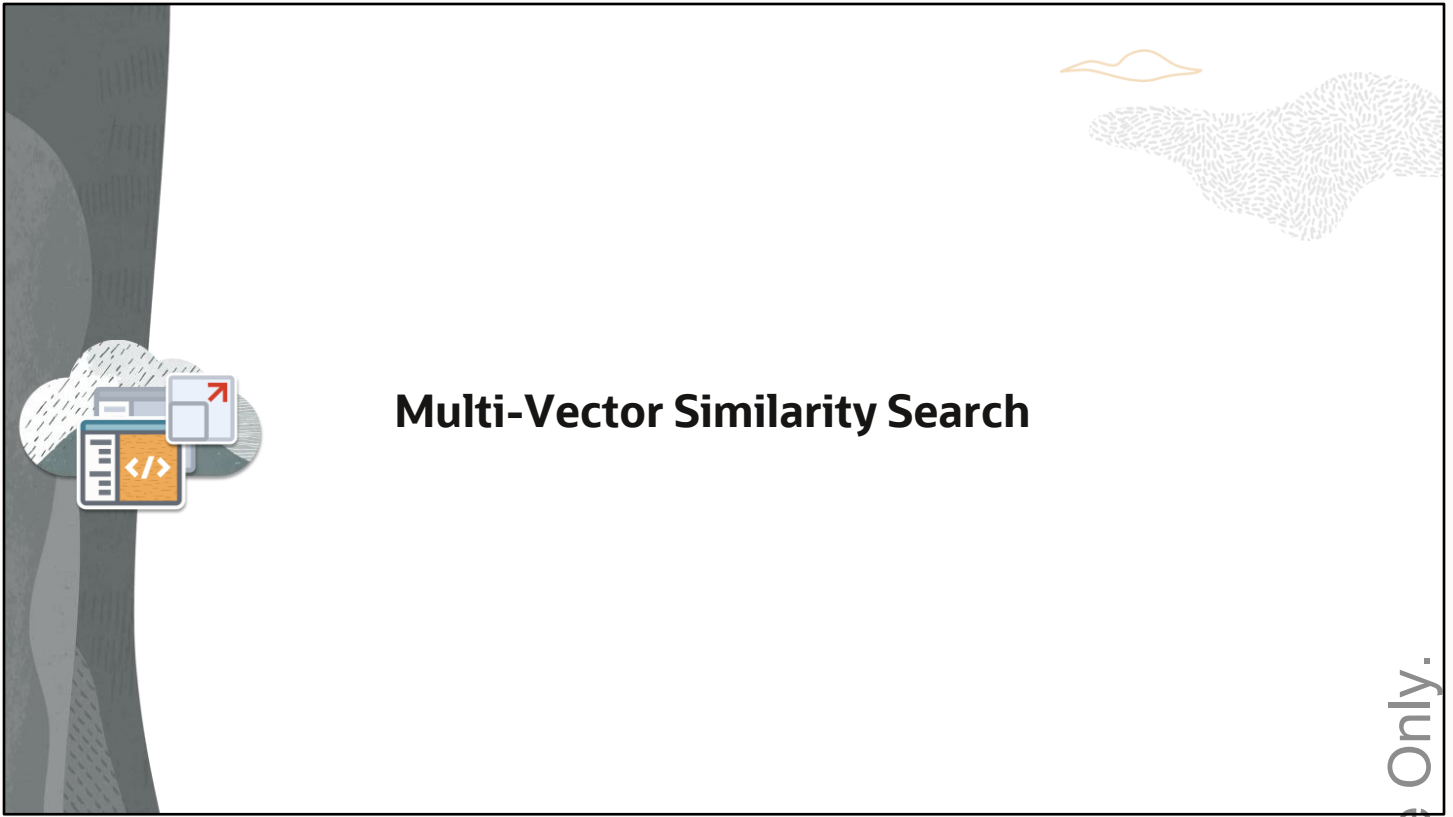
> Inverted File Flat (IVF) index

```
CREATE VECTOR INDEX galaxies_ivf_idx ON galaxies (embedding) ORGANIZATION NEIGHBOR  
PARTITIONS DISTANCE COSINE WITH TARGET ACCURACY 95;  
  
SELECT name FROM galaxies WHERE name <> 'NGC1073' ORDER BY VECTOR_DISTANCE(  
embedding, to_vector('[0,1,1,0,0]'), COSINE ) FETCH APPROXIMATE FIRST 3 ROWS ONLY;
```

[Approximate Search Using IVF \(https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/approximate-search-using-ivf.html\)](https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/approximate-search-using-ivf.html)

The example on the slide shows how you can create the Inverted File Flat (IVF) index and run an approximate search using that index.

For Instructor Use Only.
This document should not be distributed.



Multi-Vector Similarity Search

For Instructor Use Only.
This document should not be distributed.

Multi-Vector Similarity Search

- > Usually for multi-document search
- > Documents are split into chunks.
 - Chunks embedded individually into vectors
- > Uses partitions



Another major use-case of vector search is multi-vector search. Multi-vector search is typically associated with a multi-document search, where documents are split into chunks that are individually embedded into vectors.

A multi-vector search consists of retrieving top-K vector matches using grouping criteria known as partitions based on the documents' characteristics. This ability to score documents based on the similarity of their chunks to a query vector being searched is facilitated in SQL using the partitioned row limiting clause.

For Instructor Use Only.

This document should not be distributed.

Multi-Vector Similarity Search

- > Using exact similarity search:

```
SELECT bookId, paragraphId, sentence
FROM books
ORDER BY vector_distance(sentence_embedding, :sentence_query_vector)
FETCH FIRST 2 PARTITIONS BY bookId, 3 PARTITIONS BY paragraphId, 4 ROWS ONLY;
```

- > Using approximate search:

```
SELECT bookId, paragraphId, sentence
FROM books
ORDER BY vector_distance(sentence_embedding, :sentence_query_vector)
FETCH APPROXIMATE FIRST 2 PARTITIONS BY bookId, 3 PARTITIONS BY paragraphId, 4
ROWS ONLY WITH TARGET ACCURACY 90;
```

With multi-vector search, it is easier to write SQL statements to answer the following type of question:

- If they exist, what are the four best matching sentences found in the three best matching paragraphs of the two best matching books?

Imagine if each book in your database is organized into paragraphs containing sentences which have vector embedding representations. You can then answer this question using a single SQL statement such as the one in the first example above.

You can also use an approximate similarity search instead of an exact similarity search as shown in the second example above.

Note:

All the rows returned are ordered by VECTOR_DISTANCE() and not grouped by the partition clause.

Semantically, the SQL statement in the second example is interpreted as:

- Sort all records in the books table in descending order of the vector distance between the sentences and the query vector.
- For each record in this order, check its bookId and paragraphId. This record is produced if the following three conditions are met:
 1. The bookId is one of the first two distinct bookId in the sorted order.
 2. The paragraphId is one of the first three distinct paragraphId in the sorted order within the same bookId.

3. The record is one of the first four records within the same bookId and paragraphId combination.
- Otherwise, this record is filtered out.

Multi-vector similarity search is not just for documents and can be used to answer the following examples too:

- Return the top K closest matching photos but ensure that they are photos of different people.
- Find the top K songs with two or more audio segments that best match this sound snippet.

Note:

- This partition row-limiting clause extension is a generic extension of the SQL language. It does not have to apply just to vector searches.
- Multi-vector search with the partitioning row-limit clause does not use vector indexes.

For Instructor Use Only.
This document should not be distributed.



Demo

For Instructor Use Only.
This document should not be distributed.

Summary



Exact Similarity Search

Approximate Similarity Search

Multi-Vector Similarity Search

In this lesson, you have learned how to run the following basic queries on vectors:

- Exact Similarity Search
- Approximate Similarity Search
- Multi-Vector Similarity Search

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Narrowing Search Results



For Instructor Use Only.

This document should not be distributed.

Objectives



Attribute Filtering

In this lesson, you gain an understanding of narrowing search results using attribute filtering.

For Instructor Use Only.
This document should not be distributed.



Attribute Filtering

For Instructor Use Only.

This document should not be distributed.

Attribute Filtering

- > Use the WHERE clause:
 - With the VECTOR_DISTANCE() function



The `vector_distance()` function can be used in combination with other SQL WHERE predicates in order to filter the results.

For Instructor Use Only.
This document should not be distributed.

Attribute Filtering

> Sample of the data used in the examples that follow:

```
SQL> SELECT id, vsize, shape, color, v
2 FROM vt2
3 ORDER BY id;
```

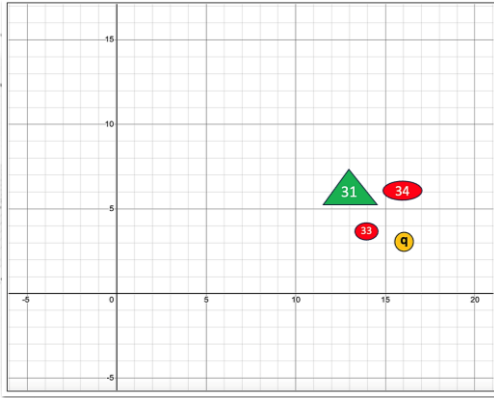
ID	VSIZE	SHAPE	COLOR	V
1	Small	Square	Green	[3.0E+000,3.0E+000]
2	Large	Triangle	Blue	[5.0E+000,3.0E+000]
3	Large	Square	Blue	[7.0E+000,3.0E+000]
4	Small	Triangle	Green	[3.0E+000,5.0E+000]
5	Medium	Oval	Red	[5.0E+000,5.0E+000]
6	Small	Square	Green	[7.0E+000,5.0E+000]
7	Large	Triangle	Blue	[3.0E+000,7.0E+000]
8	Small	Oval	Red	[5.0E+000,7.0E+000]
9	Small	Oval	Blue	[7.0E+000,7.0E+000]
21	Small	Oval	Green	[9.0E+000,-1.0E+000]
22	Medium	Triangle	Blue	[1.0E+001,-1.0E+000]
23	Small	Oval	Blue	[1.1E+001,-1.0E+000]

For Instructor Use Only.
This document should not be distributed.

Attribute Filtering

Example:

- > Look for the three closest vectors to the yellow “q” (16,3).

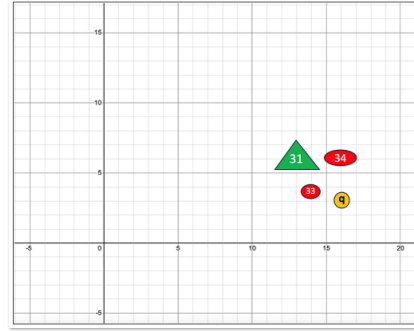


For Instructor Use Only.
This document should not be distributed.

Attribute Filtering

Example:

- > Query to look for the three closest vectors to the yellow “q” (16,3)



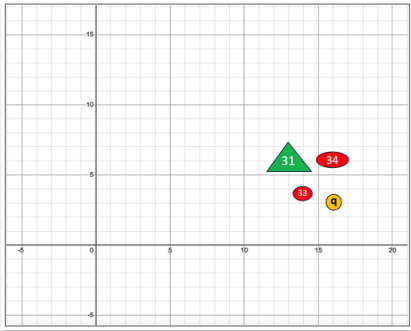
```
SELECT id, vsize, shape, color,  
       to_number(vector_distance(vector('[16, 3]'), v, EUCLIDEAN))  
distance  
FROM   vt2  
WHERE  id > 30 AND id < 40  
ORDER  BY vector_distance(vector('[16, 3]'), v, EUCLIDEAN)  
FETCH FIRST 3 ROWS ONLY;
```

For Instructor Use Only.
This document should not be distributed.

Attribute Filtering

Example:

> Results



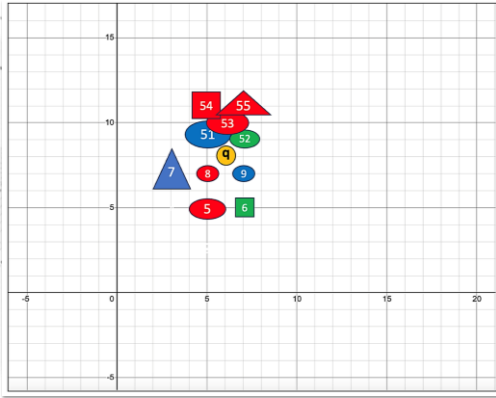
ID	VSIZE	SHAPE	COLOR	DISTANCE
33	Small	Oval	Red	2.23606801
34	Medium	Oval	Red	3
31	Large	Triangle	Green	4.2426405

For Instructor Use Only.
This document should not be distributed.

Attribute Filtering

Another example:

- > Look for the ten closest vectors to the yellow “q” (6,8).

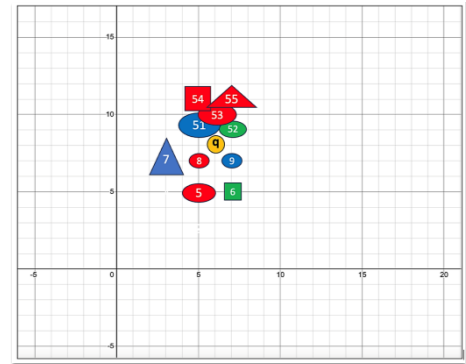


For Instructor Use Only.
This document should not be distributed.

Attribute Filtering

Another example:

- > Query to look for the ten closest vectors to the yellow “q” (16,3)



```
SELECT id, vsize, shape, color
FROM   vt2
ORDER  BY vector_distance(vector('[6, 8]'), v, EUCLIDEAN)
FETCH FIRST 10 ROWS ONLY;
```

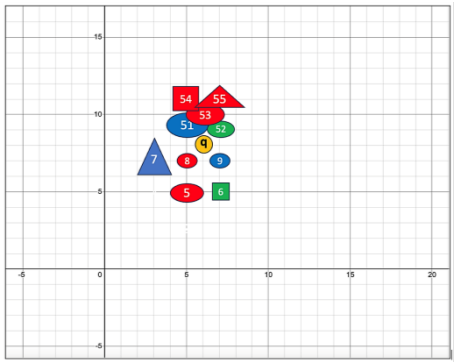
For Instructor Use Only.
This document should not be distributed.

Attribute Filtering

Another example:

> Results

ID	VSIZE	SHAPE	COLOR
8	Small	Oval	Red
52	Small	Oval	Green
51	Large	Oval	Blue
9	Small	Oval	Blue
53	Medium	Oval	Red
5	Medium	Oval	Red
55	Medium	Triangle	Red
54	Medium	Square	Red
7	Large	Triangle	Blue
6	Small	Square	Green



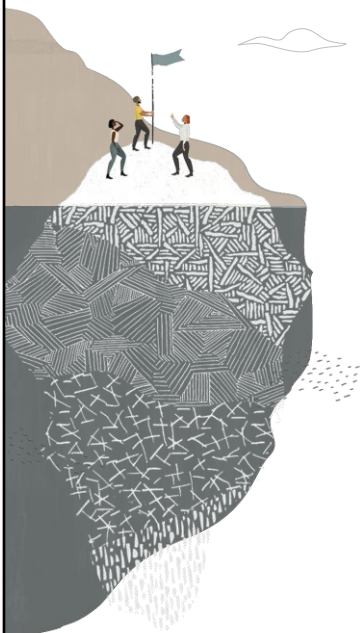
For Instructor Use Only.
This document should not be distributed.



Demo

For Instructor Use Only.
This document should not be distributed.

Summary



Attribute Filtering

In this lesson, you have learned how to narrow search results using attribute filtering.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Testing Other Distance Functions



For Instructor Use Only.

This document should not be distributed.

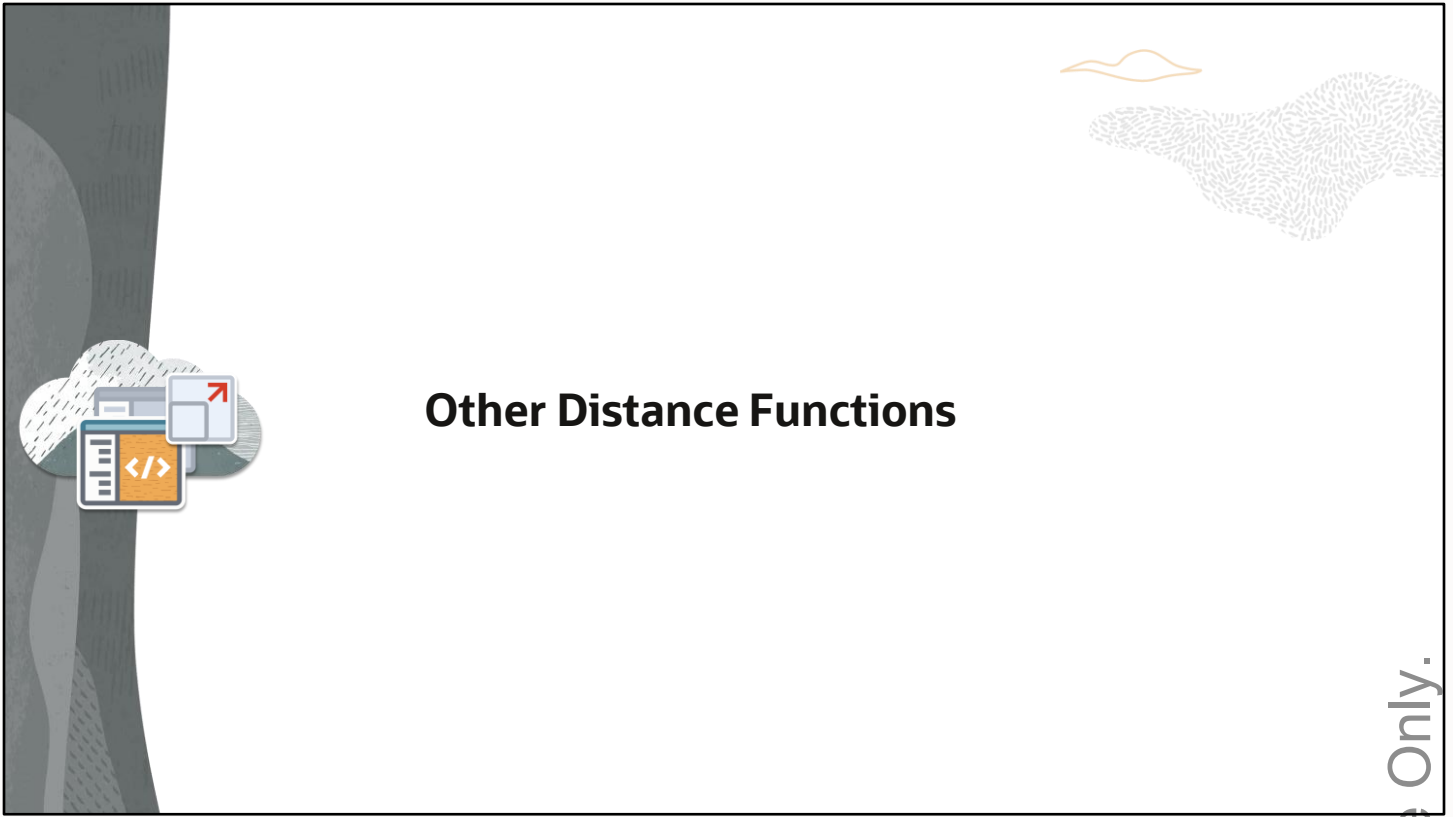
Objectives



Other Distance Functions

In this lesson, you gain an understanding of other distance functions.

For Instructor Use Only.
This document should not be distributed.



Other Distance Functions

For Instructor Use Only.
This document should not be distributed.

Other Vector Distance Functions

- > `L1_DISTANCE(v1, v2)`
- > `L2_DISTANCE(v1, v2)`
- > `COSINE_DISTANCE(v1, v2)`
- > `INNER_PRODUCT(v1, v2)`



There are four ways to find the closest vectors using other vector distance functions.

- `L1_DISTANCE (v1, v2)` = **MANHATTAN distance**
- `L2_DISTANCE (v1, v2)` = **EUCLIDEAN distance**
- `COSINE_DISTANCE (v1, v2)` = **COSINE similarity**
- `INNER_PRODUCT (v1, v2)` = **DOT product**

L1_DISTANCE

- > Look for the four closest vectors to (16,4) using L1_DISTANCE.

```
SELECT id, vsize, shape, color
FROM vt3
ORDER BY L1_DISTANCE(vector('[16, 4]'), v)
FETCH FIRST 4 ROWS ONLY;
```

ID	VSIZE	SHAPE	COLOR
33	Small	Oval	Red
34	Medium	Oval	Red
31	Large	Triangle	Green
32	Medium	Oval	Blue

`L1_DISTANCE(v1, v2)` = MANHATTAN distance

L2_DISTANCE

- > Look for the four closest vectors to (16,4) using L2_DISTANCE.

```
SELECT id, vsize, shape, color
FROM vt3
ORDER BY L2_DISTANCE(vector('[16, 4]'), v)
FETCH FIRST 4 ROWS ONLY;
```

ID	VSIZE	SHAPE	COLOR
33	Small	Oval	Red
34	Medium	Oval	Red
31	Large	Triangle	Green
32	Medium	Oval	Blue

`L2_DISTANCE(v1, v2)` = EUCLIDEAN distance

COSINE_DISTANCE

- > Look for the four closest vectors to (16,4) using COSINE_DISTANCE.

```
SELECT id, vsize, shape, color
FROM vt3
ORDER BY COSINE_DISTANCE(vector('[16, 4]'), v)
FETCH FIRST 4 ROWS ONLY;
```

ID	VSIZE	SHAPE	COLOR
33	Small	Oval	Red
34	Medium	Oval	Red
3	Large	Square	Blue
31	Large	Triangle	Green

`COSINE_DISTANCE(v1, v2)` = COSINE similarity

INNER_PRODUCT

- > Look for the four closest vectors to (16,4) using INNER_PRODUCT.

```
SELECT id, vsize, shape, color
FROM vt3
ORDER BY INNER_PRODUCT(vector('[16, 4]'), v)
FETCH FIRST 4 ROWS ONLY;
```

ID	VSIZE	SHAPE	COLOR
44	Small	Triangle	Red
41	Large	Triangle	Blue
45	Small	Oval	Red
43	Medium	Square	Blue

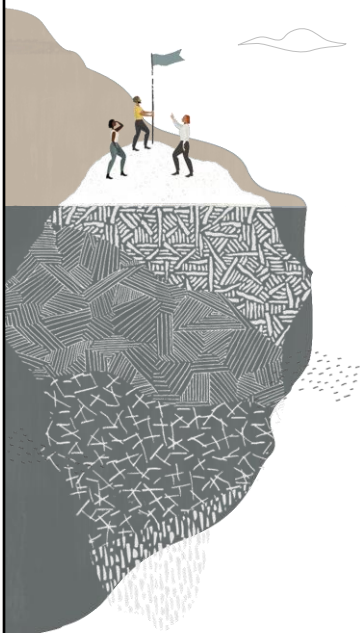
`INNER_PRODUCT(v1, v2) = DOT product`



Demo

For Instructor Use Only.
This document should not be distributed.

Summary



Other Distance Functions

In this lesson, you have gained an understanding of other distance functions.

Testing Other Vector Functions



For Instructor Use Only.

This document should not be distributed.

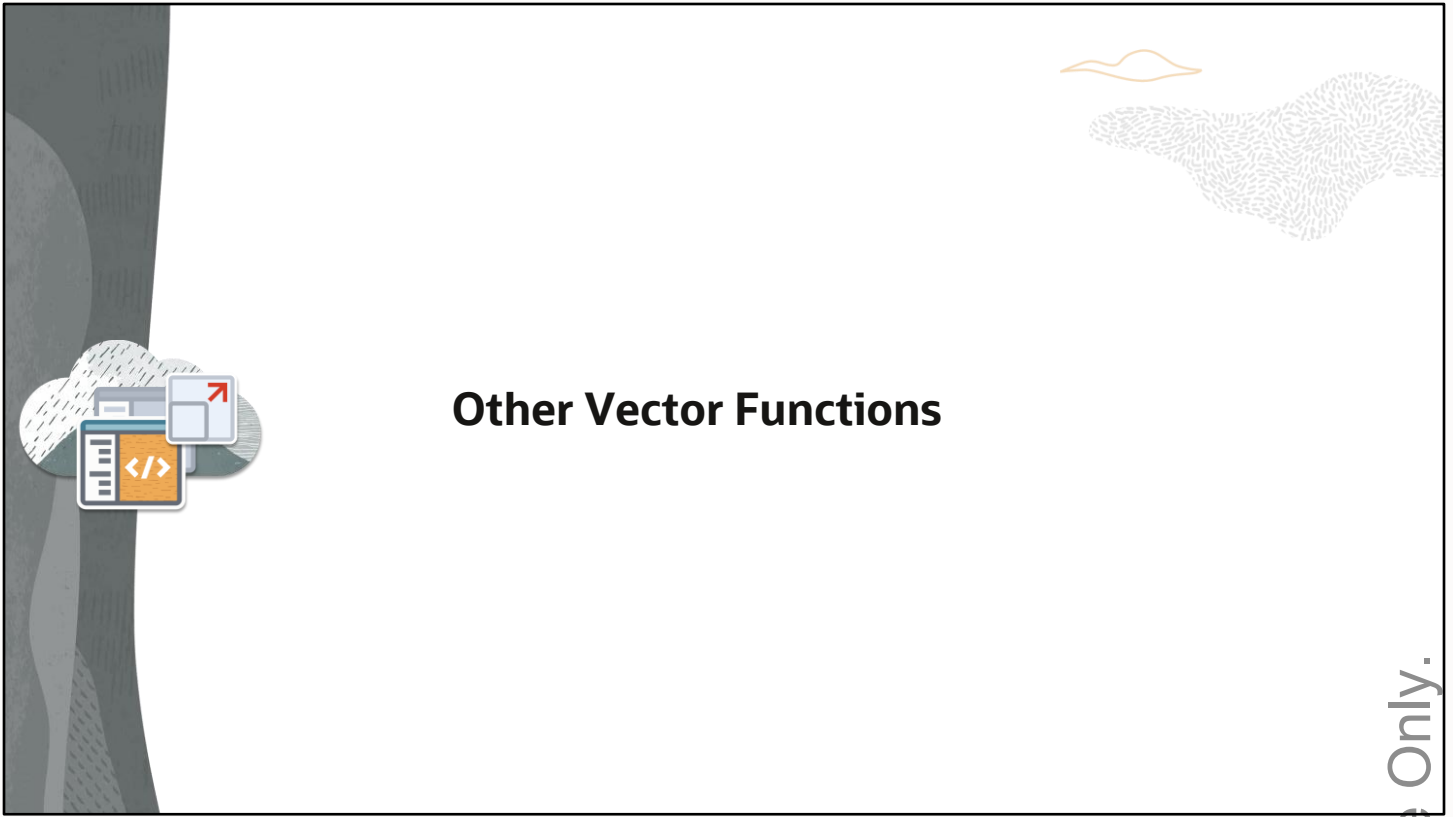
Objectives



Other Vector Functions

In this lesson, you learn about other vector functions.

For Instructor Use Only.
This document should not be distributed.



Other Vector Functions

For Instructor Use Only.
This document should not be distributed.

Other Vector Functions

- > Vector Constructors
- > Vector Serializers
- > Vector Norm
- > Vector Dimension Count
- > Vector Dimension Format



There are additional basic vector operations for Oracle AI Vector Search that involve creating, converting, and describing vectors.

For Instructor Use Only.
This document should not be distributed.

Vector Constructors

- > TO_VECTOR()
- > Converts a string or CLOB into a vector

```
SELECT to_vector('[34.6, 77.8]', 2, float32) FROM dual;
```

```
TO_VECTOR(' [34.6,77.8] ',2,FLOAT32)
-----
[3.45999985E+001,7.78000031E+001]
```

[Vector Constructors \(https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/vector-constructors.html#GUID-2B9F10B9-01E7-4C9B-A8B6-E1B7D51F737D\)](https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/vector-constructors.html#GUID-2B9F10B9-01E7-4C9B-A8B6-E1B7D51F737D)

TO_VECTOR() and VECTOR() are synonymous constructors of vectors. The functions take a string of type VARCHAR2 or CLOB as input and return a vector as output.

For Instructor Use Only.
This document should not be distributed.

Vector Constructors

> VECTOR()

- Converts a string or CLOB into a vector

```
SELECT vector(' [1.1, 2.2, 3.3]', 3, float32);
```

```
VECTOR(' [1.1,2.2,3.3]',3,FLOAT32)
```

```
-----  
[1.10000002E+000,2.20000005E+000,3.29999995E+000]
```

For Instructor Use Only.

This document should not be distributed.

Vector Serializers

- > VECTOR_SERIALIZE()
 - Converts a vector into a string or CLOB

```
SELECT vector_serialize(vector('[1.1, 2.2, 3.3]', 3, float32));
```

```
VECTOR_SERIALIZE(VECTOR('[1.1,2.2,3.3]',3,FLOAT32))
-----
[1.10000002E+000,2.20000005E+000,3.29999995E+000]
```

[Vector Serializers \(https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/vector-serializers.html#GUID-0876CB8B-8749-4A40-9B20-A27CE7415241\)](https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/vector-serializers.html#GUID-0876CB8B-8749-4A40-9B20-A27CE7415241)

FROM_VECTOR() and VECTOR_SERIALIZE() are synonymous serializers of vectors.

For Instructor Use Only.

This document should not be distributed.

Vector Serializers

- > FROM_VECTOR()
 - Converts a vector into a string or CLOB

```
SELECT from_vector(vector(' [1.1, 2.2, 3.3] ', 3, float32));
```

```
FROM_VECTOR(VECTOR(' [1.1,2.2,3.3] ',3,FLOAT32))  
-----  
[1.10000002E+000,2.20000005E+000,3.29999995E+000]
```

For Instructor Use Only.
This document should not be distributed.

Vector Norm

> VECTOR_NORM()

- Returns the Euclidean norm of a vector
 - The distance from the origin to the vector

```
SELECT vector_norm(vector(' [4, 3]', 2, float32) );
```

```
VECTOR_NORM(VECTOR(' [4, 3]', 2, FLOAT32))  
-----  
5.0E+000
```

VECTOR_NORM (https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/vector_norm.html)

The VECTOR_NORM() function returns the Euclidean norm of a vector ($\text{SQRT}(\text{SUM}((x_i - y_i)^2))$) in the format of BINARY_DOUBLE.

For Instructor Use Only.
This document should not be distributed.

Vector Dimension Count

- > VECTOR_DIMENSION_COUNT()
 - Returns the number of dimensions of a vector

```
SELECT vector_dimension_count(vector(' [34.6, 77.8]', 2, float64));
```

```
VECTOR_DIMENSION_COUNT(VECTOR(' [34.6, 77.8]', 2, FLOAT64))  
-----  
2
```

[VECTOR_DIMENSION_COUNT](https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/vector_dimension_count.html) (https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/vector_dimension_count.html)

The VECTOR_DIMENSION_COUNT() function returns the number of dimensions of a vector in the format of an Oracle number.

For Instructor Use Only.
This document should not be distributed.

Vector Dimension Format

- > VECTOR_DIMENSION_FORMAT()
 - Returns the storage format of the vector

```
SELECT vector_dimension_format(vector('[34.6, 77.8]', 2, float64));
```

```
VECTOR_DIMENSION_FORMAT(VECTOR('[34.6,77.8]',2,FLO  
-----  
FLOAT64
```

[VECTOR_DIMENSION_FORMAT](https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/vector_dimension_format.html) (https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/vector_dimension_format.html)

The VECTOR_DIMENSION_FORMAT() returns the storage format of the vector. It returns a VARCHAR2, which can be one of the following values: 'INT8', 'FLOAT32', or 'FLOAT64'.

For Instructor Use Only.
This document should not be distributed.

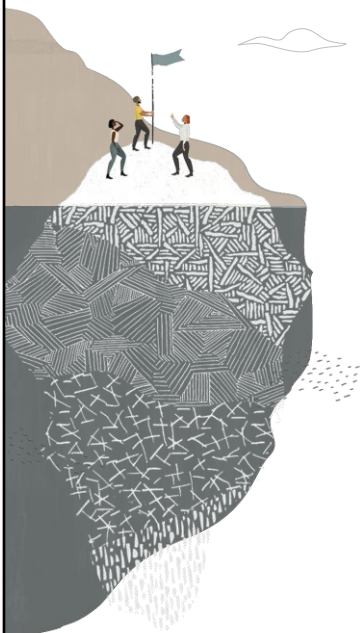


Demo

For Instructor Use Only.

This document should not be distributed.

Summary



Other Vector Functions

In this lesson, you have gained an understanding of other vector functions.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Course Conclusion

Oracle AI Vector Search Fundamentals



For Instructor Use Only.
This document should not be distributed.

Learning Summary

Vector Search
Overview



Benefits



VECTOR Datatype



Vector Embedding
Models



Vector Workflow



For Instructor Use Only.

This document should not be distributed.

Learning Summary

Basic Queries
on Vectors



Vector Indexes



Vector Memory



Vector DML
and DDL



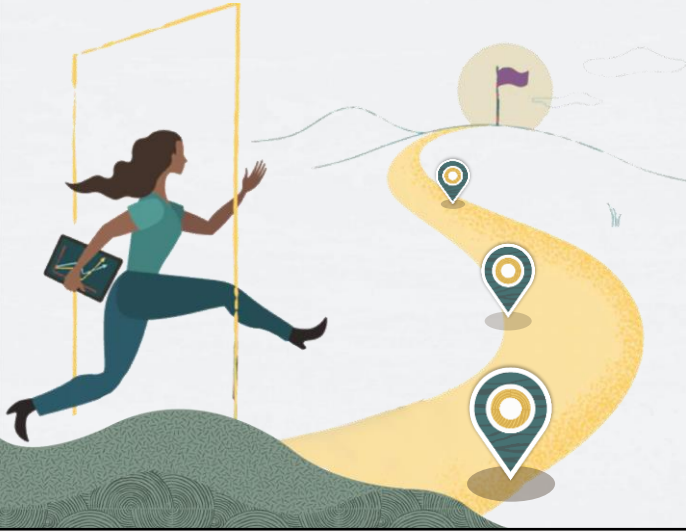
Vector Functions
and Filtering



For Instructor Use Only.

This document should not be distributed.

Thank You!



**We appreciate you
taking this course!**

For Instructor Use Only.
This document should not be distributed.